C++

Complete Notes

From your first program to modern C++20 / C++23

Basics | Pointers | OOP | Templates | STL | Lambdas | Smart pointers | Move semantics | Exceptions | File I/O | Concurrency | Interview prep

```cpp
#include <iostream>

int main() {
    std::cout << "Hello, C++!\n";
    return 0;
}
```

15 chapters | diagrams, worked examples, cheat sheets and interview questions

Contents

1. Getting Started with C++ . . . 3
2. Variables, Types and Operators . . . 7
3. Control Flow . . . 12
4. Functions . . . 15
5. Arrays, Strings, Pointers, References . . . 18
6. Classes and Objects . . . 23
7. Inheritance and Polymorphism . . . 27
8. Templates and Generic Programming . . . 31
9. The STL: Containers . . . 34
10. Iterators, Algorithms and Lambdas . . . 38
11. Memory Safety: RAII, Smart Pointers, Move Semantics . . . 42
12. Exceptions and File I/O . . . 46
13. Modern C++ (11 to 23) at a Glance . . . 50
14. Concurrency and Performance . . . 53
15. Best Practices, Mistakes and Interview Questions . . . 56

IN PLAIN WORDS

How to use these notes: read a chapter in order, type every example yourself, and do the practice problems in Chapter 15. Boxes marked **Careful!** are traps; boxes marked **Remember this** are the ideas that come up in interviews. All code compiles with `g++ -std=c++20`, and the examples use `std::` explicitly so you can see where every name comes from.

TIP

Tools you need: a compiler (GCC, Clang or MSVC), a text editor or IDE (VS Code, CLion), and optionally CMake. Compiler Explorer (godbolt.org) lets you try snippets in the browser with no setup.

CHAPTER 1

Getting Started with C++

What it is, how it runs, and your first working program

Before the syntax, it helps to know what C++ is for and what actually happens between typing code and seeing output.

1.1 What is C++?

C++ is a general-purpose, compiled, statically typed language. Bjarne Stroustrup began it at Bell Labs around 1979 as "C with Classes", because he wanted the speed and low-level control of C together with the organisation tools of Simula. It was renamed C++ in 1983 (the `++` is a joke on C's increment operator) and has been standardised by ISO ever since.

Standard	Year	What it brought
C++98 / C++03	1998 / 2003	Templates, STL, exceptions, namespaces
C++11	2011	`auto`, lambdas, move semantics, smart pointers, `nullptr`, threads
C++14	2014	Generic lambdas, `make_unique`, return type deduction
C++17	2017	`optional`, `variant`, structured bindings, `filesystem`, `if constexpr`
C++20	2020	Concepts, ranges, coroutines, modules, `<=>`
C++23	2023	`std::expected`, `std::print`, `flat_map`, deducing this

Where C++ is used

- Game engines (Unreal), graphics and simulation
- Operating systems, browsers, databases
- Trading systems and anything latency sensitive
- Embedded devices, robotics, automotive
- Compilers, VMs, and the runtimes of other languages

Why people choose it

- **Speed.** There is no garbage collector or interpreter between you and the machine.
- **Control.** You decide how memory is laid out and when it is freed.
- **Abstraction without cost.** Classes, templates and algorithms usually compile down to the same code you would write by hand.
- **Huge ecosystem.** Decades of libraries, tools and code.

MY OWN NOTE

The price of all that power is that C++ lets you shoot yourself in the foot. Most of these notes are about learning the habits (RAII, const, smart pointers) that keep the gun pointed away from your foot.

1.2 How a C++ program becomes an executable

A computer cannot run `.cpp` text directly. A toolchain turns your source into machine code in four stages. Knowing the stages makes error messages far less mysterious, because each stage has its own typical errors.

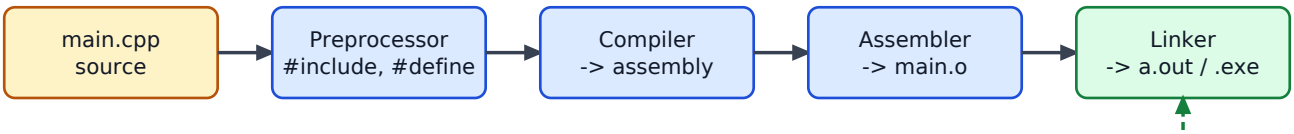


Figure 1.1 - The build pipeline

1. **Preprocessing** handles lines starting with `#`. `#include` pastes a header file in; `#define` does text substitution. Output: one big translation unit.
2. **Compiling** checks syntax and types, then translates to assembly. Syntax and type errors appear here.
3. **Assembling** turns assembly into an object file (`.o` or `.obj`) holding machine code with unresolved references.
4. **Linking** joins object files and libraries, resolving references. "undefined reference to ..." errors come from this stage.

Terminal

```
g++ -std=c++17 -Wall -Wextra -O2 main.cpp -o app   # compile + link in one go
./app                                                 # run it

# the long way, one stage at a time
g++ -E main.cpp -o main.i    # preprocess only
g++ -S main.i   -o main.s    # to assembly
g++ -c main.s   -o main.o    # to object file
g++ main.o      -o app       # link
```

> **TIP**
>
> Always compile with `-Wall -Wextra`. The warnings catch real bugs: uninitialised variables, unused values, missing returns, signed/unsigned comparisons.

1.3 Your first program

hello.cpp

```
#include <iostream>        // brings in cin, cout, endl

int main() {                // execution always starts here
    std::cout << "Hello, C++!" << std::endl;
    return 0;               // 0 means success
}
```

OUTPUT

```
Hello, C++!
```

Reading it line by line:

- `#include <iostream>` - angle brackets mean a standard library header; quotes (`"mine.h"`) mean your own file.

- `int main()` - the entry point. It returns an `int` to the operating system. Reaching the closing brace without a `return` is allowed for `main` only and counts as `return 0`.
- `std::cout` - the standard output stream. The `std::` prefix says it lives in namespace `std`.
- `<<` - the insertion operator. It pushes values into the stream, and you can chain it.
- `std::endl` - writes a newline *and* flushes the buffer. A plain `'\n'` is faster when you do not need the flush.

1.4 Anatomy of a C++ file

structure.cpp

```
// 1. includes
#include <iostream>
#include <string>

// 2. using declarations (be modest, see below)
using std::string;

// 3. global constants, type definitions, function declarations
const double PI = 3.14159;
string greet(const string& name);

// 4. main
int main() {
    std::cout << greet("Asha") << "\n";
}

// 5. function definitions
string greet(const string& name) {
    return "Hello, " + name + "!";
}
```

Namespaces and `using`

A namespace is a named container that prevents name clashes. The whole standard library lives in `std`. You can write `std::cout` every time, import one name with `using std::cout;`, or import everything with `using namespace std;`. The last one is fine in a ten-line practice file, but in headers or big projects it can silently cause collisions, so most professionals avoid it.

```
namespace physics { double g = 9.81; }
namespace moon    { double g = 1.62; }

std::cout << physics::g << " vs " << moon::g;   // no clash
```

1.5 Comments

```
// single-line comment

/* multi-line
   comment */

/// documentation style comment (used by Doxygen)
```

IN PLAIN WORDS

Good comments explain *why*, not *what*. `i++; // add one to i` is noise. `i++; // skip the header row` is useful.

1.6 Input and output

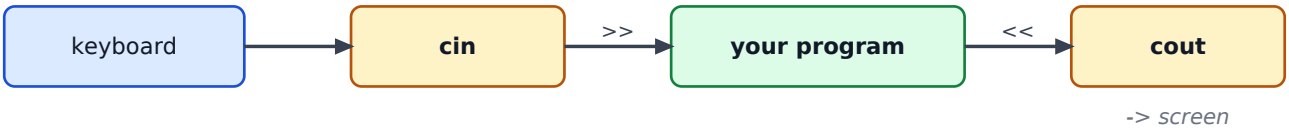


Figure 1.2 - Streams: data flows through cin and cout

io.cpp

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string name;
    int age;
    cout << "Name: ";
    getline(cin, name);          // reads a whole line, spaces included
    cout << "Age: ";
    cin >> age;                  // stops at whitespace
    cout << name << " is " << age << " years old\n";
}
```

CAREFUL!

Mixing `cin >> x` and `getline` is a classic trap. After `cin >> age` the newline is still in the buffer, so the next `getline` returns an empty string. Fix it with `cin.ignore(numeric_limits<streamsize>::max(), '\n');` before the `getline`.

Formatting output

```
#include <iomanip>
double x = 3.14159265;
cout << fixed << setprecision(2) << x << "\n";   // 3.14
cout << setw(8) << setfill('*') << 42 << "\n";   // ******42
cout << boolalpha << true << "\n";                // true
cout << hex << 255 << " " << dec << 255;          // ff 255
```

Making cin/cout faster

In competitive programming and large-input tasks, add these two lines at the start of `main`. They detach C++ streams from C's `stdio` and untie `cin` from `cout`:

```
std::ios::sync_with_stdio(false);
std::cin.tie(nullptr);
```

1.7 Chapter recap

- C++ is compiled: preprocess, compile, assemble, link.
- `main` is the entry point; a program is a set of functions and types.
- Streams (`cin`, `cout`) do I/O; `<<` and `>>` chain.
- Namespaces prevent name clashes; avoid `using namespace std;` in headers.
- Turn on warnings and a modern standard (`-std=c++17` or later).

CHAPTER 2

Variables, Types and Operators

Storing data, doing arithmetic, and not getting bitten by conversions

Every value in C++ has a type, fixed at compile time. The type decides how much memory it uses and which operations make sense.

2.1 Fundamental types

Type	Typical size	Holds	Example
`bool`	1 byte	true / false	`bool ok = true;`
`char`	1 byte	one character / small int	`char c = 'A';`
`short`	2 bytes	small integers	`short s = -300;`
`int`	4 bytes	whole numbers (about +/-2.1 billion)	`int n = 42;`
`long long`	8 bytes	big integers (about +/-9.2e18)	`long long big = 1e12;`
`float`	4 bytes	~7 digits of precision	`float f = 1.5f;`
`double`	8 bytes	~15 digits of precision	`double d = 3.14;`
`unsigned`	same as signed	non-negative only	`unsigned u = 7;`

IN PLAIN WORDS

Sizes are only guaranteed as minimums, so they can vary by platform. Use `sizeof(int)` to check, or the fixed-width types from `<cstdint>` such as `int32_t`, `uint8_t`, `int64_t` when exact size matters.

sizes.cpp

```
#include <iostream>
#include <climits>
int main() {
    std::cout << "int bytes: " << sizeof(int) << "\n";
    std::cout << "int max  : " << INT_MAX << "\n";
    std::cout << "ll  max  : " << LLONG_MAX << "\n";
}
```

OUTPUT

```
int bytes: 4
int max  : 2147483647
ll  max  : 9223372036854775807
```

2.2 Declaring and initialising

```
int a = 5;        // copy initialisation
int b(5);         // direct initialisation
int c{5};         // brace (uniform) initialisation - preferred
int d{};          // zero-initialised to 0
int e;            // UNINITIALISED: garbage value, do not read

int x = 3.9;      // allowed, silently truncates to 3
int y{3.9};       // ERROR: braces forbid narrowing, which is a good thing
```

REMEMBER THIS

Brace initialisation `{}` refuses lossy conversions. That single feature prevents many silent bugs, so prefer it.

auto - let the compiler deduce

```
auto i = 10;        // int
auto pi = 3.14;     // double
auto s = std::string("hi");
auto it = v.begin();   // saves typing a very long iterator type
```

`auto` is not dynamic typing. The type is still fixed at compile time; you just let the compiler work it out. Use it when the type is obvious from the right-hand side or absurdly long to write.

2.3 Constants

```
const int MAX = 100;        // cannot change after init
constexpr int SQUARE5 = 5 * 5; // evaluated at compile time
constexpr double TAU = 2 * 3.14159;

// MAX = 200;   // ERROR
```

Keyword	Meaning
`const`	Read-only after initialisation. The value may be known only at run time.
`constexpr`	Must be computable at compile time. Can be used for array sizes, template arguments.
`#define`	Old-style text substitution with no type or scope. Avoid in modern code.

2.4 Operators

Arithmetic

```
int a = 17, b = 5;
a + b   // 22
a - b   // 12
a * b   // 85
a / b   // 3   <- integer division truncates
a % b   // 2   <- remainder (integers only)
17 / 5.0  // 3.4 <- one double operand makes the whole thing double
```

CAREFUL!

Integer division is the most common beginner surprise: `1 / 2` is `0`, not `0.5`. Write `1.0 / 2` or `1 / 2.0`.

Increment and decrement

```
int i = 5;
int a = i++;   // a = 5, then i = 6   (post: use, then increase)
int b = ++i;   // i = 7, then b = 7  (pre:  increase, then use)
```

Comparison and logical

Operator	Meaning	Operator	Meaning
`==`	equal	`&&`	logical AND (short-circuits)
`!=`	not equal	`||`	logical OR (short-circuits)
`< >`	less / greater	`!`	logical NOT
`<= >=`	less-eq / greater-eq	`<=>`	three-way compare (C++20)

Short-circuiting means `a && b` never evaluates `b` if `a` is false. That is why `if (p != nullptr && p->ok())` is safe.

Bitwise operators

```
unsigned x = 0b1100, y = 0b1010;
x & y    // 0b1000  AND
x | y    // 0b1110  OR
x ^ y    // 0b0110  XOR
~x       // flips every bit
x << 2   // 0b110000  shift left = multiply by 4
x >> 1   // 0b110     shift right = divide by 2

// common tricks
bool odd   = n & 1;
bool bit3  = (n >> 3) & 1;
n |= (1 << 4);     // set bit 4
n &= ~(1 << 4);    // clear bit 4
```

Assignment and compound assignment

```
x += 3;  x -= 2;  x *= 4;  x /= 2;  x %= 5;
x <<= 1; x &= mask; x |= flag; x ^= toggle;
```

Ternary operator

```
int bigger = (a > b) ? a : b;
std::string s = (score >= 40) ? "pass" : "fail";
```

Operator precedence (high to low, abridged)

Level	Operators
1	`:: () [] . -> x++ x--`
2	`! ~ ++x --x` unary `- *` (deref) `&` (address) `sizeof new`
3	`* / %`
4	`+ -`
5	`<< >>`
6	`< <= > >=`
7	`== !=`
8-10	`&` then `^` then `|`
11-12	`&&` then `||`
13	`?: = +=` ... (right to left)
14	`,`

> **TIP**
>
> When in doubt, add parentheses. Readers will thank you; the compiler does not mind.

2.5 Type conversion (casting)

Conversions happen implicitly (the compiler does it) or explicitly (you ask for it). Implicit ones usually move toward the "bigger" type: `int + double` becomes `double`. Dangerous ones go the other way or mix signed and unsigned.

```
double d = 7 / 2;              // 3.0   (int division first!)
double e = 7 / 2.0;            // 3.5
int    n = static_cast<int>(3.99);   // 3

unsigned u = 1;
int s = -1;
if (s < u) { }   // FALSE! -1 converts to a huge unsigned number
```

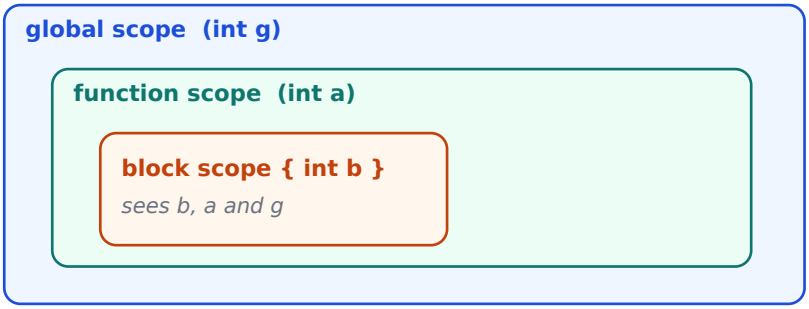
Cast	Use it for
`static_cast<T>(x)`	normal, well-defined conversions between related types (most casts)
`const_cast<T>(x)`	adding or removing `const` (rare, risky)
`dynamic_cast<T>(x)`	safe down-casts in polymorphic hierarchies (checked at run time)
`reinterpret_cast<T>(x)`	reinterpret raw bits, e.g. pointer to integer (very risky)
`(T)x`	C-style cast. Avoid: it silently picks the most dangerous option.

2.6 Overflow, precision and gotchas

- Signed integer overflow is **undefined behaviour**. `INT_MAX + 1` may do anything. Use a wider type (`long long`) or check first.
- Unsigned overflow wraps around (it is defined), so `0u - 1` becomes the largest unsigned value.
- Floating point cannot store most decimals exactly: `0.1 + 0.2 == 0.3` is **false**. Compare with a tolerance: `std::abs(a - b) < 1e-9`.
- Never compare floats with `==` unless you know they came from the same exact computation.

```
#include <cmath>
double a = 0.1 + 0.2;
bool exact = (a == 0.3);                    // false
bool close = std::fabs(a - 0.3) < 1e-9;     // true
```

2.7 Scope and lifetime



inner scopes can see outward, never the other way round

Figure 2.1 - Nested scopes: inner blocks can see outward

```
int g = 1;                  // global: lives for the whole program
void f() {
    int a = 2;              // local: dies when f returns
    {
        int b = 3;          // block scope: dies at the closing brace
        int a = 99;         // shadows outer 'a' - legal but confusing
    }
    static int calls = 0;  // static local: created once, remembers its value
    ++calls;
}
```

2.8 Chapter recap

- Pick the right type; use `long long` for large counts and `double` for real numbers.
- Initialise everything. Prefer `{}` to block narrowing.
- `const` / `constexpr` express intent and let the compiler help you.
- Watch integer division, signed/unsigned mixing and float equality.
- Prefer `static_cast` over C-style casts.

CHAPTER 3

Control Flow

Decisions and loops: making programs do more than one thing

Programs become interesting once they can choose and repeat. C++ gives you the usual toolkit, with a few modern twists.

3.1 if / else

```
int marks = 72;
if (marks >= 90) {
    std::cout << "A";
} else if (marks >= 75) {
    std::cout << "B";
} else if (marks >= 60) {
    std::cout << "C";
} else {
    std::cout << "Try again";
}
```

Any value converts to a condition: zero is false, everything else is true. That is handy but also the source of the famous `=` versus `==` bug:

CAREFUL!

`if (x = 5)` assigns 5 to `x` and is always true. Compilers warn about it with `-Wall`. Some people write `if (5 == x)` so that a typo becomes an error.

if with an initialiser (C++17)

```
if (auto it = m.find(key); it != m.end()) {
    std::cout << it->second;     // `it` exists only inside this if/else
} else {
    std::cout << "missing";
}
```

3.2 switch

```
char grade = 'B';
switch (grade) {
    case 'A': std::cout << "Excellent"; break;
    case 'B':
    case 'C': std::cout << "Good";      break;   // B and C share code
    default:  std::cout << "Unknown";
}
```

REMEMBER THIS

Forgetting `break` makes execution *fall through* into the next case. It works only with integers, chars and enums, not strings. In C++17 you can mark intentional fall-through with `[[fallthrough]];`.

3.3 Loops

while and do-while

```
int n = 5;
while (n > 0) {              // checks first, may run zero times
    std::cout << n-- << ' ';
}                            // 5 4 3 2 1

int x;
do {                         // runs at least once
    std::cout << "Enter positive: ";
    std::cin >> x;
} while (x <= 0);
```

for

```
for (int i = 0; i < 5; ++i) {
    std::cout << i << ' ';       // 0 1 2 3 4
}

for (int i = 10; i > 0; i -= 3) { }     // 10 7 4 1
for (;;) { /* infinite loop, leave with break */ }
```

Range-based for (C++11)

```
std::vector<int> v{3, 1, 4, 1, 5};

for (int x : v)         std::cout << x;   // copy of each element
for (int& x : v)        x *= 2;           // reference: modifies the vector
for (const auto& x : v) std::cout << x;   // read-only, no copy: best default

for (auto [k, val] : myMap)  { }         // C++17 structured bindings
```

> **TIP**
>
> For anything bigger than an `int`, loop with `const auto&` so you do not copy every string or object.

break, continue, goto

```
for (int i = 1; i <= 10; ++i) {
    if (i % 2 == 0) continue;   // skip evens
    if (i > 7)      break;      // stop the loop entirely
    std::cout << i << ' ';      // 1 3 5 7
}
```

`goto` exists but is almost never needed; a function with an early `return` is usually cleaner.

3.4 Nested loops and patterns

pattern.cpp

```
// right triangle of stars
for (int row = 1; row <= 4; ++row) {
    for (int col = 1; col <= row; ++col)
        std::cout << '*';
    std::cout << '\n';
}
```

```
OUTPUT
*
**
***
****
```

The outer loop picks the row, the inner loop does the work for that row. Time cost multiplies: two nested loops of size n do about n x n steps.

3.5 Common loop patterns

```
// sum and average
int sum = 0;
for (int x : v) sum += x;
double avg = static_cast<double>(sum) / v.size();

// maximum
int mx = v[0];
for (int x : v) if (x > mx) mx = x;

// linear search
int pos = -1;
for (size_t i = 0; i < v.size(); ++i)
    if (v[i] == target) { pos = i; break; }
```

3.6 Worked example: prime check and FizzBuzz

loops.cpp

```
bool isPrime(int n) {
    if (n < 2) return false;
    for (int d = 2; d * d <= n; ++d)    // only need to test up to sqrt(n)
        if (n % d == 0) return false;
    return true;
}

for (int i = 1; i <= 15; ++i) {
    if      (i % 15 == 0) std::cout << "FizzBuzz ";
    else if (i % 3  == 0) std::cout << "Fizz ";
    else if (i % 5  == 0) std::cout << "Buzz ";
    else                  std::cout << i << ' ';
}
```

```
OUTPUT
1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
```

MY OWN NOTE

Check `i % 15` first in FizzBuzz. If you test `% 3` first, the "FizzBuzz" branch can never run. Order of conditions matters in an else-if chain.

3.7 Chapter recap

- `if / else if / else` for choices; `switch` for many integer cases (remember `break`).
- `while` checks first, `do-while` runs once at least, `for` for counted loops.
- Range-based `for` with `const auto&` is the cleanest way to read a container; `break` leaves a loop, `continue` skips an iteration.

CHAPTER 4

Functions

Breaking a program into reusable, testable pieces

A function gives a name to a piece of work. Good functions are small, do one thing, and have clear inputs and outputs.

4.1 Declaring and defining

```
// declaration (prototype): tells the compiler the signature
int add(int a, int b);

int main() {
    std::cout << add(3, 4);      // 7
}

// definition: the actual body
int add(int a, int b) {
    return a + b;
}
```

The compiler reads top to bottom, so a function must be declared before it is used. In real projects declarations live in header files (`.h`) and definitions in source files (`.cpp`).

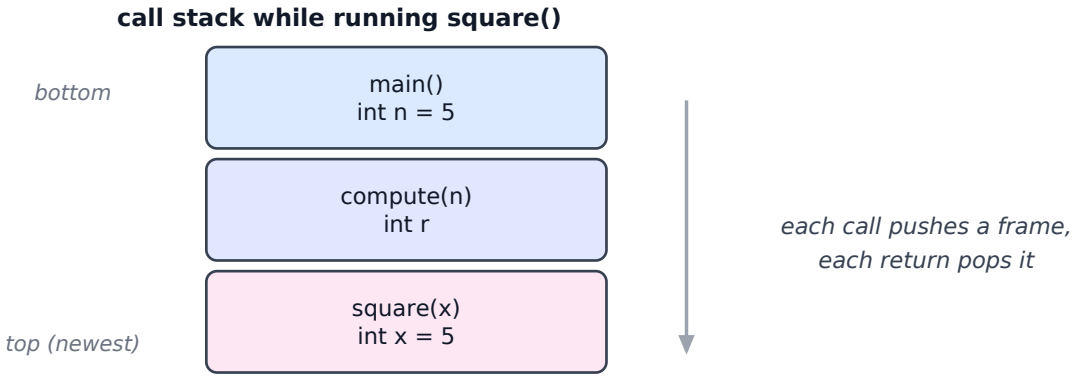


Figure 4.1 - Every call pushes a stack frame holding its parameters and locals

4.2 Passing arguments

Style	Syntax	What happens	Use when
By value	`void f(int x)`	function gets a copy	small types (int, double, char)
By reference	`void f(int& x)`	function works on the original	you want to modify the caller's variable
By const reference	`void f(const string& s)`	no copy, no modification	big read-only objects (default for strings, vectors)
By pointer	`void f(int* p)`	address is copied, can be null	optional argument or C-style APIs
By rvalue reference	`void f(string&& s)`	binds to temporaries	taking ownership / move semantics

```
void byValue(int x)      { x = 99; }      // caller's variable unchanged
void byRef(int& x)       { x = 99; }      // caller's variable becomes 99
void readOnly(const std::string& s) { std::cout << s; }

int a = 1, b = 1;
byValue(a);   // a is still 1
byRef(b);     // b is now 99
```

REMEMBER THIS

Rule of thumb: cheap-to-copy types by value, everything else by `const&`, and use a non-const `&` only for out-parameters you truly need to change.

4.3 Return values

```
int square(int x) { return x * x; }

void hello() { std::cout << "hi"; }      // void: returns nothing

std::pair<int,int> minmax(const std::vector<int>& v);   // return two values
auto [lo, hi] = minmax(v);                              // C++17 unpack
```

CAREFUL!

Never return a reference or pointer to a local variable. The local dies when the function ends, leaving a dangling reference. `int& bad() { int x = 5; return x; }` is a bug.

Returning big objects by value is fine and usually free: the compiler applies *return value optimisation* (RVO) or moves the object instead of copying it. Do not write `std::move` on a returned local; it prevents the optimisation.

4.4 Default arguments

```
void connect(std::string host, int port = 80, bool secure = false);

connect("example.com");            // port 80, not secure
connect("example.com", 443);       // port 443
connect("example.com", 443, true);
```

Defaults must be the *rightmost* parameters and are given once, in the declaration (usually the header).

4.5 Function overloading

Several functions may share a name if their parameter lists differ. The compiler picks the best match at compile time. The return type alone is not enough to tell them apart.

```
int    area(int side)           { return side * side; }
double area(double r)           { return 3.14159 * r * r; }
int    area(int w, int h)       { return w * h; }

area(4);        // int version
area(2.5);      // double version
area(3, 5);     // two-parameter version
```

4.6 inline, constexpr and static

- `inline` - a hint (today mostly a linker rule) that the function may be defined in headers without breaking the one-definition rule.

- `constexpr` - the function can run at compile time when given constant arguments.

```
constexpr int factorial(int n) { return n <= 1 ? 1 : n * factorial(n - 1); }
constexpr int f5 = factorial(5);    // computed during compilation = 120
std::array<int, factorial(4)> arr;  // size 24, known at compile time
```

4.7 Recursion

A recursive function calls itself on a smaller problem. It always needs a **base case** that stops the recursion and a step that moves toward it.

recursion.cpp

```
int fact(int n) {
    if (n <= 1) return 1;          // base case
    return n * fact(n - 1);        // smaller problem
}

int fib(int n) {                    // simple but slow: O(2^n)
    return n < 2 ? n : fib(n - 1) + fib(n - 2);
}

int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }   // Euclid
```

MY OWN NOTE

`fib(40)` with the naive version takes seconds because it recomputes the same values millions of times. Store results (memoisation) or use a loop and it becomes instant. Recursion is elegant, but always ask how deep it can go: each call uses stack memory and a very deep recursion crashes with a stack overflow.

4.8 Function pointers, std::function and lambdas

```
int apply(int x, int (*op)(int)) { return op(x); }   // function pointer
int dbl(int v) { return v * 2; }
apply(5, dbl);                                          // 10

#include <functional>
std::function<int(int)> f = [](int v) { return v + 1; };
f(9);                                                   // 10
```

Lambdas are covered fully in Chapter 10. For now: a lambda is an unnamed function you can write inline, handy for callbacks and algorithms.

4.9 Good function design

- One function, one job. If you need "and" in its name, split it.
- Keep them short: if it does not fit on a screen, ask if it can be broken up.
- Mark parameters `const` when you do not change them, and prefer returning a value over an out-parameter.

4.10 Chapter recap

- Declare before use; pass by value for small types, `const&` for large read-only, `&` to modify.
- Overloading chooses by parameters; defaults go on the right.
- Recursion needs a base case; never return a reference to a local.

CHAPTER 5

Arrays, Strings, Pointers, References

How C++ sees memory, and how to point at it safely

This is the chapter that separates C++ from most languages. Understand it and everything later (classes, smart pointers, the STL) makes sense.

5.1 Arrays

An array is a fixed number of values of one type stored side by side in memory. Because they are contiguous, jumping to element `i` is instant.

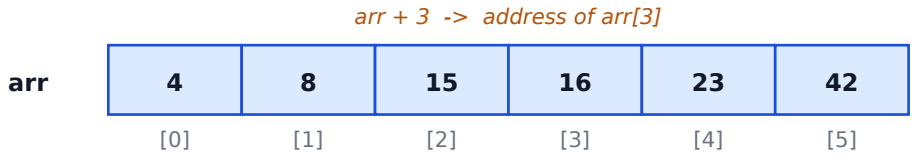


Figure 5.1 - An array is a contiguous block; indexes start at 0

```
int a[5] = {4, 8, 15, 16, 23};     // size 5
int b[]   = {1, 2, 3};             // size deduced = 3
int c[5]  = {};                    // all zeros

a[0] = 100;                        // first element
int last = a[4];                   // last element
// a[5] = 1;                       // UNDEFINED BEHAVIOUR: no bounds checking!

int grid[3][4] = {};               // 2-D array: 3 rows, 4 columns
grid[1][2] = 7;
```

CAREFUL!

C-style arrays do not check bounds, do not know their own size once passed to a function (they decay to a pointer), and cannot be copied with `=`. Prefer `std::array` (fixed size) or `std::vector` (growable).

```
#include <array>
std::array<int, 5> arr{4, 8, 15, 16, 23};
arr.size();      // 5
arr.at(2);       // 15, throws std::out_of_range if bad index
arr.front();  arr.back();
for (int x : arr) { }
```

5.2 Strings

C++ has two kinds of strings. The C kind is a char array ending in a zero byte (`"hello"` is really `h e l l o \0`). The C++ kind, `std::string`, manages its own memory and is what you should use almost always.

```
#include <string>
std::string s = "Hello";
s += ", World";                 // append
s.size();                      // 12 (same as s.length())
```

```
s[0];                          // 'H'
s.substr(7, 5);                // "World"
s.find("World");               // 7  (std::string::npos if absent)
s.replace(0, 5, "Howdy");
s.insert(5, "!!!");
s.erase(0, 2);
s.empty();  s.clear();

std::string t = "apple";
t < "banana";                  // true: lexicographic comparison
std::to_string(42);            // "42"
std::stoi("123");              // 123
std::stod("3.14");             // 3.14
```

Iterating and transforming

```
#include <algorithm>
#include <cctype>
std::string s = "Hello World";
std::transform(s.begin(), s.end(), s.begin(),
               [](unsigned char c){ return std::toupper(c); });
// s == "HELLO WORLD"
std::reverse(s.begin(), s.end());
int vowels = std::count_if(s.begin(), s.end(),
    [](char c){ return std::string("AEIOUaeiou").find(c) != std::string::npos; });
```

Splitting with stringstream

```
#include <sstream>
std::istringstream in("the quick brown fox");
std::string word;
while (in >> word) std::cout << word << "|";     // the|quick|brown|fox|
```

std::string_view (C++17)

A `string_view` is a cheap, non-owning window onto characters that live elsewhere. It is ideal for function parameters that only read text, because it accepts `std::string`, literals and char arrays without copying. Never keep one longer than the string it views.

```
void shout(std::string_view sv) { /* read only */ }
shout("literal");
shout(someStdString);
```

5.3 Pointers

Every variable lives at some address in memory. A **pointer** is a variable whose value *is* an address. Two operators matter: `&x` means "address of x" and `*p` means "the thing p points to".

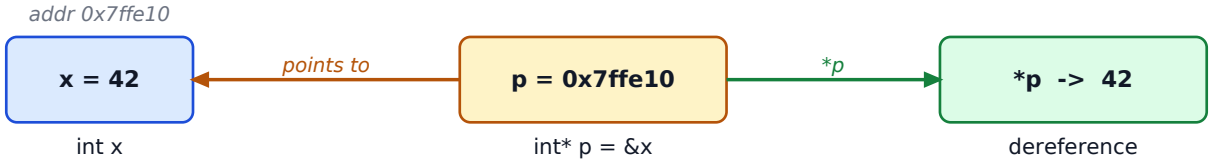


Figure 5.2 - A pointer holds an address; dereferencing follows it

```
int x = 42;
int* p = &x;          // p holds the address of x

std::cout << p;       // address, e.g. 0x7ffe10
std::cout << *p;      // 42
```

```
*p = 100;             // changes x through the pointer
std::cout << x;       // 100

int* none = nullptr;  // points at nothing
if (none) { }         // false: always test before dereferencing
```

CAREFUL!

Dereferencing `nullptr`, an uninitialised pointer or a freed pointer is undefined behaviour and the most common cause of crashes. Always initialise pointers: to a real address or to `nullptr`.

Pointer arithmetic

```
int a[] = {10, 20, 30, 40};
int* p = a;           // array name decays to the address of a[0]
*(p + 2);             // 30   same as a[2]
++p;                  // now points at a[1]; moved by sizeof(int) bytes
p[1];                 // 30   indexing works on pointers too
```

Adding 1 to a pointer moves it by the size of the type it points to, not by one byte. That is why `a[i]` and `*(a + i)` mean exactly the same thing.

const and pointers (read right to left)

Declaration	Read it as	Can change data?	Can repoint?
`const int* p`	pointer to const int	no	yes
`int* const p`	const pointer to int	yes	no
`const int* const p`	const pointer to const int	no	no

Pointers to pointers and arrays of pointers

```
int x = 5;
int* p = &x;
int** pp = &p;        // pointer to a pointer
**pp = 9;             // x is now 9

const char* names[] = {"Ana", "Ben", "Chi"};   // array of pointers to text
```

5.4 References

A reference is an alias: another name for an existing variable. Unlike a pointer it must be initialised, can never be re-seated to something else, and cannot be null. Use the name normally; no `*` needed.

```
int x = 10;
int& r = x;       // r IS x
r = 20;           // x becomes 20
int y = 5;
r = y;            // copies y's VALUE into x; r still refers to x
```

	Pointer	Reference
Can be null	yes	no
Can change target	yes	no
Must be initialised	no (but should be)	yes
Syntax to use	`*p`, `p->`	just the name
Typical use	optional things, ownership, arrays, C APIs	parameters, aliases, returning elements

REMEMBER THIS

Choose a reference when something must exist. Choose a pointer (or `std::optional`) when it might not.

5.5 Dynamic memory: new and delete

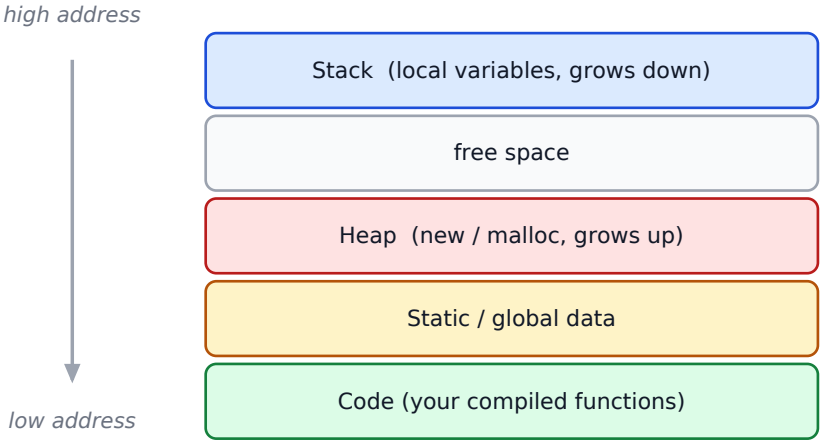


Figure 5.3 - The process memory map. Locals live on the stack, new objects on the heap

Stack variables vanish automatically when their scope ends, but the stack is small and fixed. When you need memory that outlives a function, or whose size is unknown until run time, you ask the heap with `new` and must give it back with `delete`.

```
int* p = new int(7);          // one int on the heap
delete p;                      // give it back
p = nullptr;                   // avoid a dangling pointer

int* arr = new int[100];       // array of 100 ints
arr[0] = 1;
delete[] arr;                  // arrays need delete[] (not delete)
```

CAREFUL!

Three classic bugs: **memory leak** (forgot `delete`), **double free** (deleted twice), **use after free** (used the pointer after `delete`). The cure is not to be more careful. The cure is to stop writing raw `new`/`delete` and use smart pointers and containers (Chapter 11).

Stack vs heap at a glance

	Stack	Heap
Allocation	automatic, very fast	manual / smart pointer, slower
Size	small (about 1-8 MB)	limited only by RAM
Lifetime	until scope ends	until you free it
Failure mode	stack overflow	leaks, fragmentation, bad_alloc

5.6 Chapter recap

- Prefer `std::array`, `std::vector`, `std::string` over raw arrays and char*.

- `&x` is the address, `*p` follows a pointer; always initialise pointers.
- References are non-null aliases and great for parameters.
- Every `new` needs a matching `delete`, which is exactly why RAII and smart pointers exist.

CHAPTER 6

Classes and Objects

Bundling data with the functions that work on it

A class is a blueprint for a new type. It lets you model things from the real problem, such as an Account, a Car or a Vector2D, and guard their rules.

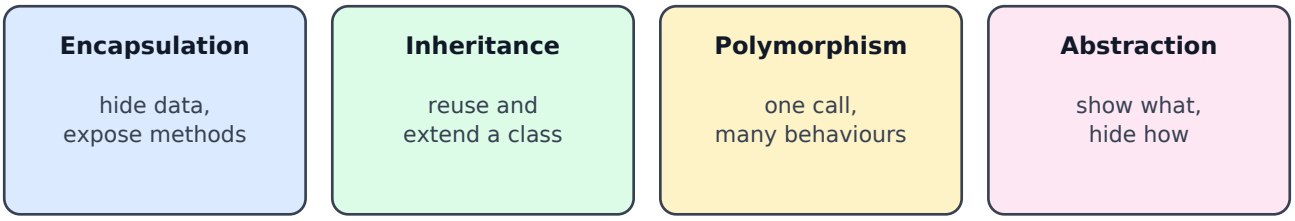


Figure 6.1 - The four pillars of object-oriented programming

6.1 Defining a class

account.cpp

```
#include <string>
#include <iostream>

class Account {
private:                            // hidden from outside code
    std::string owner_;
    double balance_ = 0.0;          // default member initialiser

public:                             // the interface
    Account(std::string owner, double opening)
        : owner_(std::move(owner)), balance_(opening) {}   // member initialiser list

    void deposit(double amt) {
        if (amt > 0) balance_ += amt;
    }
    bool withdraw(double amt) {
        if (amt <= 0 || amt > balance_) return false;
        balance_ -= amt;
        return true;
    }
    double balance() const { return balance_; }          // const: does not modify
    const std::string& owner() const { return owner_; }
};

int main() {
    Account a("Meera", 500);
    a.deposit(250);
    a.withdraw(100);
    std::cout << a.owner() << " has " << a.balance();     // Meera has 650
    // a.balance_ = 1e9;   // ERROR: private
}
```

class vs struct

They are identical except for the default access: `class` members are private by default, `struct` members are public. By convention use `struct` for plain bundles of data and `class` when there are invariants to protect.

Encapsulation: why private?

If `balance_` were public, any code anywhere could set it to a negative number. Making it private forces every change through `deposit` and `withdraw`, so the rules are checked in one place. Later you can change how the data is stored without breaking anyone's code.

6.2 Constructors

A constructor runs when an object is created and has the same name as the class, with no return type. Its job is to put the object into a valid state.

```
class Point {
public:
    Point() : x_(0), y_(0) {}                       // default constructor
    Point(double x, double y) : x_(x), y_(y) {}   // parameterised
    explicit Point(double both) : x_(both), y_(both) {}   // explicit: no silent conversion
    Point(const Point& other) = default;          // copy constructor
private:
    double x_, y_;
};

Point a;            // default
Point b(1, 2);      // parameterised
Point c{3, 4};      // brace init
Point d = b;        // copy
```

REMEMBER THIS

Members are initialised in the order they are *declared in the class*, not the order in the initialiser list. Keep the two in the same order to avoid confusing bugs. Prefer the initialiser list to assigning inside the body: it constructs members directly instead of default-constructing then overwriting.

Delegating constructors

```
class Rect {
public:
    Rect() : Rect(1, 1) {}                      // delegates to the one below
    Rect(int w, int h) : w_(w), h_(h) {}
private:
    int w_, h_;
};
```

6.3 Destructors

The destructor (`~ClassName`) runs automatically when an object goes out of scope or is deleted. It is where you release anything the object acquired: memory, files, sockets, locks.

```
class FileHandle {
    FILE* f_;
public:
    explicit FileHandle(const char* path) : f_(std::fopen(path, "r")) {}
    ~FileHandle() { if (f_) std::fclose(f_); }      // always runs
    FileHandle(const FileHandle&) = delete;
    FileHandle& operator=(const FileHandle&) = delete;
};
```

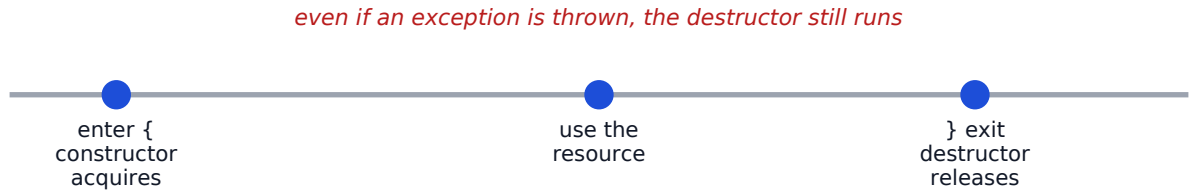


Figure 6.2 - RAII: acquire in the constructor, release in the destructor

6.4 The Rule of Three / Five / Zero

If a class owns a resource (raw memory, a file), the compiler-generated copy would copy the pointer, not the thing it points to, leaving two objects that both delete it. Then you need to define the special members yourself.

Rule	You write	When
Rule of 3	destructor, copy constructor, copy assignment	class manages a resource (C++98 thinking)
Rule of 5	the three above + move constructor + move assignment	same, plus you want cheap moves
Rule of 0	**none** of them	hold members that already manage themselves (`vector`, `string`, `unique_ptr`). **Aim for this.**

```
class Buffer {
    int* data_; size_t n_;
public:
    explicit Buffer(size_t n) : data_(new int[n]{}), n_(n) {}
    ~Buffer() { delete[] data_; }
    Buffer(const Buffer& o) : data_(new int[o.n_]), n_(o.n_) {            // deep copy
        std::copy(o.data_, o.data_ + n_, data_);
    }
    Buffer& operator=(Buffer o) {                    // copy-and-swap
        std::swap(data_, o.data_); std::swap(n_, o.n_); return *this;
    }
    Buffer(Buffer&& o) noexcept : data_(o.data_), n_(o.n_) {      // steal
        o.data_ = nullptr; o.n_ = 0;
    }
};
// With std::vector<int> as the member, you would need none of this.
```

6.5 this, const methods, static members

```
class Counter {
    int n_ = 0;
    static int total_;            // ONE copy shared by all Counter objects
public:
    Counter& inc() { ++n_; ++total_; return *this; }   // *this allows chaining
    int get() const { return n_; }                     // promises not to modify *this
    static int total() { return total_; }              // no object needed
};
int Counter::total_ = 0;          // static data member defined once, outside

Counter c;
c.inc().inc().inc();              // chained calls
std::cout << c.get() << Counter::total();   // 3 3
```

- `this` is a pointer to the current object; inside methods `n_` means `this->n_`.

- A `const` method can be called on const objects; mark everything that does not change state as `const`.
- `static` members belong to the class, not to any object.

6.6 Operator overloading

You can give operators meaning for your own types so that code reads naturally. Keep the usual meaning: `+` should add, not print.

```
struct Vec2 {
    double x{}, y{};
    Vec2 operator+(const Vec2& o) const { return {x + o.x, y + o.y}; }
    Vec2 operator*(double k)      const { return {x * k, y * k}; }
    bool operator==(const Vec2& o) const { return x == o.x && y == o.y; }
    Vec2& operator+=(const Vec2& o) { x += o.x; y += o.y; return *this; }
};

// stream output must be a free function
std::ostream& operator<<(std::ostream& os, const Vec2& v) {
    return os << "(" << v.x << ", " << v.y << ")";
}

Vec2 a{1, 2}, b{3, 4};
std::cout << a + b * 2;          // (7, 10)
```

> **TIP**
>
> C++20: declare `auto operator<=>(const T&) const = default;` and the compiler generates all six comparison operators for you.

6.7 friend, nested types, enum class

```
class Secret {
    int code_ = 1234;
    friend void audit(const Secret&);     // this function may see private members
};
void audit(const Secret& s) { std::cout << s.code_; }

enum class Color { Red, Green, Blue };     // scoped enum: no implicit int conversion
Color c = Color::Green;
switch (c) { case Color::Red: break; default: break; }
```

Use `enum class` over plain `enum`. Its names do not leak into the surrounding scope and it will not silently convert to an integer.

6.8 Chapter recap

- Class = data + behaviour; keep data private and expose a small interface.
- Constructors create valid objects; destructors clean up. Use initialiser lists.
- Prefer the **Rule of Zero**; if you must manage a resource, follow the Rule of Five.
- Mark non-mutating methods `const`. Use `explicit` on single-argument constructors.
- Overload operators only when the meaning is natural.

CHAPTER 7

Inheritance and Polymorphism

Building families of related types, and choosing behaviour at run time

Inheritance lets one class build on another. Polymorphism lets code written for the general type work correctly on every specific one.

7.1 Inheritance basics

```
class Animal {
protected:                      // visible to this class and its children
    std::string name_;
public:
    explicit Animal(std::string n) : name_(std::move(n)) {}
    void eat() const { std::cout << name_ << " eats\n"; }
};

class Dog : public Animal {     // "Dog is an Animal"
public:
    explicit Dog(std::string n) : Animal(std::move(n)) {}   // call the base constructor
    void bark() const { std::cout << name_ << " says woof\n"; }
};

Dog d("Rex");
d.eat();     // inherited
d.bark();    // its own
```

Access	Same class	Derived class	Outside
`public`	yes	yes	yes
`protected`	yes	yes	no
`private`	yes	no	no

Construction runs from the top: the base constructor first, then the derived one. Destruction runs in reverse. Use public inheritance only for a true *is-a* relationship; for *has-a* use a member (composition).

MY OWN NOTE

"Favour composition over inheritance." A `Car` *has an* `Engine`; it is not one. Deep inheritance trees become brittle because a change in the base ripples through every child.

7.2 Virtual functions and polymorphism

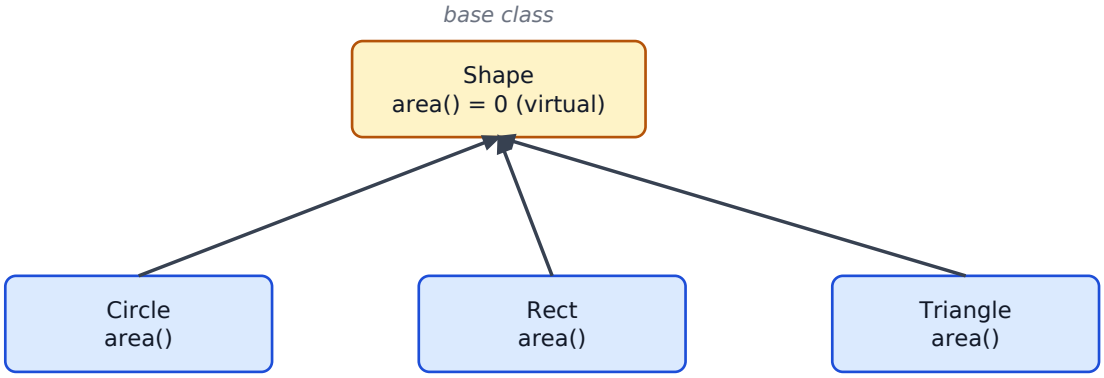


Figure 7.1 - A hierarchy: each shape overrides area()

shapes.cpp

```
class Shape {
public:
    virtual double area() const = 0;     // pure virtual: no body, makes Shape abstract
    virtual ~Shape() = default;         // virtual destructor: ESSENTIAL
};

class Circle : public Shape {
    double r_;
public:
    explicit Circle(double r) : r_(r) {}
    double area() const override { return 3.14159 * r_ * r_; }
};

class Rect : public Shape {
    double w_, h_;
public:
    Rect(double w, double h) : w_(w), h_(h) {}
    double area() const override { return w_ * h_; }
};

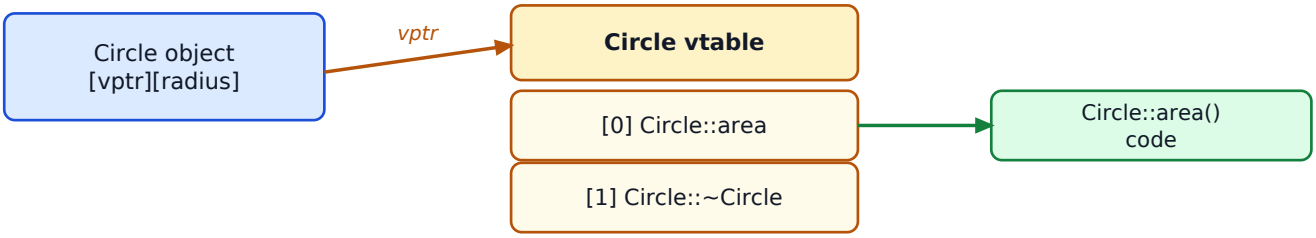
int main() {
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Circle>(2));
    shapes.push_back(std::make_unique<Rect>(3, 4));
    for (const auto& s : shapes)
        std::cout << s->area() << "\n";     // right version chosen at run time
}
```

OUTPUT

```
12.5664
12
```

The loop calls `area()` through a `Shape` pointer, yet each object runs its own version. That is **run-time (dynamic) polymorphism**. It needs three ingredients: a base class with `virtual` functions, derived classes that override them, and calls made through a pointer or reference.

How it works: the vtable



s->area() looks up slot [0] at run time

Figure 7.2 - Each polymorphic object carries a hidden pointer to its class's table of functions

For every class with virtual functions the compiler builds a table of function addresses (the *vtable*). Each object stores a hidden pointer (*vptr*) to its class's table. A virtual call looks up the slot and jumps there. The cost is one extra indirection per call and 8 bytes per object, which is small but not zero.

override, final, and the virtual destructor

- `override` makes the compiler check that you really are overriding a base function. Always write it; it catches signature typos.
- `final` forbids further overriding (on a function) or deriving (on a class).
- **Base classes must have a virtual destructor.** Otherwise `delete basePtr;` skips the derived destructor and leaks.

> **CAREFUL!**
>
> Object slicing: `Shape s = Circle(2);` copies only the `Shape` part and throws away the rest, and polymorphism is lost. Hold polymorphic objects by pointer or reference, never by value.

7.3 Abstract classes and interfaces

A class with at least one pure virtual function (`= 0`) cannot be instantiated; it only defines a contract. A class with *only* pure virtual functions and no data acts as an interface.

```
struct Logger {
    virtual void log(const std::string& msg) = 0;
    virtual ~Logger() = default;
};
struct ConsoleLogger : Logger {
    void log(const std::string& m) override { std::cout << m << '\n'; }
};
struct FileLogger : Logger {
    void log(const std::string& m) override { /* write to file */ }
};

void run(Logger& out) { out.log("started"); }   // works with any logger
```

7.4 Static vs dynamic binding

	Static (compile time)	Dynamic (run time)
Chosen by	declared type of the expression	actual type of the object
Needs	ordinary functions, overloads, templates	`virtual` + pointer/reference
Speed	can be inlined, fastest	one indirect call
Flexibility	types fixed in code	new types plug in later

7.5 dynamic_cast and RTTI

```
Shape* s = getShape();
if (auto* c = dynamic_cast<Circle*>(s)) {     // returns nullptr if s is not a Circle
    std::cout << "radius-specific work";
}
std::cout << typeid(*s).name();               // run-time type information
```

7.6 Multiple inheritance and the diamond

```
struct Flyer   { void fly()  {} };
struct Swimmer { void swim() {} };
struct Duck : Flyer, Swimmer {};       // fine when bases are independent

struct A { int v; };
struct B : virtual A {};               // virtual inheritance: share ONE A
struct C : virtual A {};
struct D : B, C {};                    // D has a single A::v, no ambiguity
```

Without `virtual`, `D` would contain two separate `A` subobjects and `d.v` would be ambiguous. In practice, multiple inheritance of *interfaces* is common and fine; multiple inheritance of data-carrying classes is best avoided.

7.7 Polymorphism without inheritance

Inheritance is not the only way. **Templates** give compile-time polymorphism (next chapter), `std::variant` + `std::visit` give a closed set of alternatives, and `std::function` wraps any callable. Choose the lightest tool that fits.

```
using Shape = std::variant<Circle, Rect>;
double totalArea(const std::vector<Shape>& v) {
    double t = 0;
    for (const auto& s : v)
        t += std::visit([](const auto& x) { return x.area(); }, s);
    return t;
}
```

7.8 Chapter recap

- Public inheritance = is-a; otherwise use composition.
- Use `virtual` + `override`; give base classes a `virtual` destructor.
- Hold polymorphic objects through pointers/references (`unique_ptr<Base>`) to avoid slicing.
- Abstract classes (`= 0`) define interfaces; prefer templates or `variant` when you do not need open-ended extension.

CHAPTER 8

Templates and Generic Programming

Write the code once, let the compiler make a version for every type

Templates are how C++ achieves reuse without losing speed. The entire STL is built from them.

8.1 Function templates

```
template <typename T>
T maxOf(T a, T b) {
    return (a > b) ? a : b;
}

maxOf(3, 7);            // T = int
maxOf(2.5, 1.5);        // T = double
maxOf<std::string>("a", "b");   // T given explicitly
```



Figure 8.1 - The compiler instantiates a separate function for each type used

Templates are resolved at compile time. Nothing is looked up when the program runs, so there is no speed penalty. The trade-off is longer compile times and, historically, very long error messages.

IN PLAIN WORDS

`maxOf(3, 2.5)` fails: the compiler sees `int` and `double` and cannot decide what `T` is. Cast one argument or specify `maxOf<double>(3, 2.5)`.

8.2 Class templates

stack.cpp

```
template <typename T>
class Stack {
    std::vector<T> items_;
public:
    void push(const T& v) { items_.push_back(v); }
    void pop()            { items_.pop_back(); }
    T&   top()            { return items_.back(); }
    bool empty() const    { return items_.empty(); }
    size_t size() const   { return items_.size(); }
};

Stack<int> s;
s.push(1); s.push(2);
Stack<std::string> names;       // a completely separate type

Stack t{};                      // C++17: class template argument deduction (CTAD)
std::pair p{1, 2.5};            // deduced pair<int,double>
```

> **TIP**
>
> Template code normally lives entirely in the header file, because the compiler needs the full definition at the point where it creates each instance.

8.3 Multiple parameters, defaults, non-type parameters

```
template <typename K, typename V = int>
struct Entry { K key; V value; };
Entry<std::string> e{"a", 1};          // V defaults to int

template <typename T, std::size_t N>   // N is a compile-time constant
struct FixedArray {
    T data[N];
    constexpr std::size_t size() const { return N; }
};
FixedArray<int, 8> fa;                 // this is how std::array works
```

8.4 Specialisation

Sometimes one type needs different code. A *full specialisation* supplies a custom version for a specific type.

```
template <typename T>
struct Printer { static void show(const T& v) { std::cout << v; } };

template <>
struct Printer<bool> {                       // special case for bool
    static void show(bool v) { std::cout << (v ? "yes" : "no"); }
};
```

8.5 Variadic templates

A parameter pack lets a template take any number of arguments. This is how `std::make_unique`, `std::tuple` and `printf`-like functions are written type-safely.

```
template <typename... Args>
void print(const Args&... args) {
    ((std::cout << args << ' '), ...);      // C++17 fold expression
    std::cout << '\n';
```

```
}
print(1, "two", 3.0, 'c');                  // 1 two 3 c

template <typename... Ts>
auto sum(Ts... xs) { return (xs + ...); }   // fold over +
sum(1, 2, 3, 4);                            // 10
```

8.6 Concepts (C++20)

Before C++20 a template accepted any type and failed deep inside with a wall of errors if the type was unsuitable. **Concepts** let you state the requirements up front and get a clear message.

```
#include <concepts>

template <typename T>
concept Number = std::integral<T> || std::floating_point<T>;

template <Number T>
T twice(T x) { return x * 2; }

twice(21);        // ok
twice(1.5);       // ok
// twice("hi");   // clear error: 'const char*' does not satisfy Number

auto add(std::integral auto a, std::integral auto b) { return a + b; }   // abbreviated form
```

8.7 if constexpr and type traits

```
#include <type_traits>
template <typename T>
std::string describe(const T& v) {
    if constexpr (std::is_integral_v<T>)           return "integer " + std::to_string(v);
    else if constexpr (std::is_floating_point_v<T>) return "float " + std::to_string(v);
    else                                            return "something else";
}  // the branches that do not apply are not even compiled
```

8.8 Chapter recap

- `template <typename T>` makes a blueprint; the compiler stamps out one copy per type.
- Put templates in headers; expect longer builds.
- Specialise for special types, use parameter packs for variable arguments.
- In C++20 use concepts to state requirements; in C++17 use `if constexpr`.

CHAPTER 9

The STL: Containers

Ready-made, tested, fast data structures

The Standard Template Library gives you containers (where data lives), iterators (how you walk it) and algorithms (what you do with it). Learn to reach for these before writing your own.

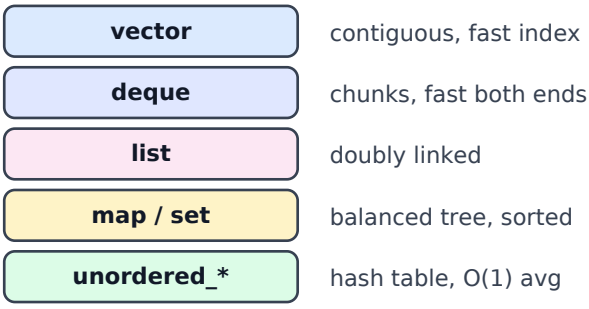


Figure 9.1 - The main containers and what each is good at

9.1 std::vector - your default container

A `vector` is a resizable array. Elements sit contiguously in memory, which makes indexing and iteration extremely fast and cache friendly. If you are unsure which container to use, start here.

```
#include <vector>
std::vector<int> v;                 // empty
std::vector<int> w(5, 0);           // five zeros
std::vector<int> x{3, 1, 4, 1, 5};  // initialiser list

v.push_back(10);                    // append, amortised O(1)
v.emplace_back(20);                 // construct in place
v.pop_back();                       // remove last
v[0];   v.at(0);                    // unchecked / bounds-checked
v.front(); v.back();
v.size(); v.empty(); v.capacity();
v.reserve(1000);                    // allocate ahead: avoids regrowth
v.insert(v.begin() + 1, 99);        // O(n): shifts later elements
v.erase(v.begin());                 // O(n)
v.clear();
```

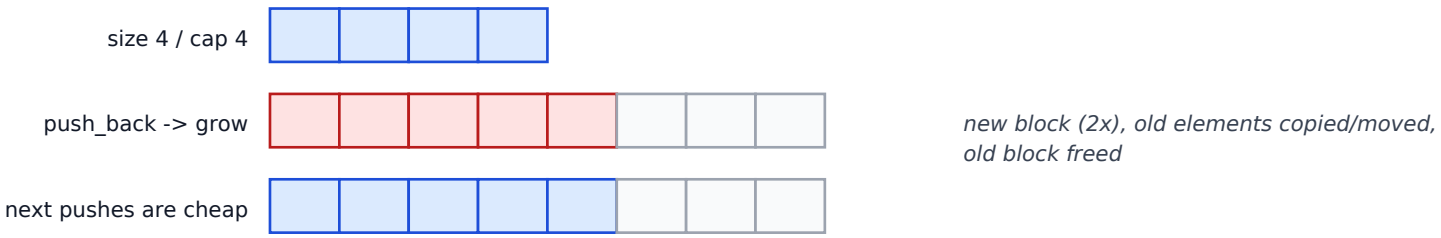


Figure 9.2 - When full, a vector allocates a bigger block and moves everything over

CAREFUL!

Inserting or growing can **invalidate** iterators, pointers and references to elements, because the data may move. Do not hold them across a `push_back`.

The erase-remove idiom

```
std::vector<int> v{1, 2, 3, 2, 4, 2};
v.erase(std::remove(v.begin(), v.end(), 2), v.end());   // v = 1 3 4

std::erase(v, 3);                                        // C++20 shortcut
std::erase_if(v, [](int n) { return n % 2 == 0; });      // C++20
```

9.2 Other sequence containers

Container	Strength	Weakness	Use when
`array<T,N>`	no heap, size known at compile time	fixed size	small fixed collections
`deque<T>`	O(1) push/pop at both ends	not one contiguous block	queues, sliding windows
`list<T>`	O(1) insert/erase anywhere with an iterator, stable references	slow to traverse, extra memory per node	frequent splicing in the middle (rare)
`forward_list<T>`	smallest linked list	forward only	memory-tight singly linked

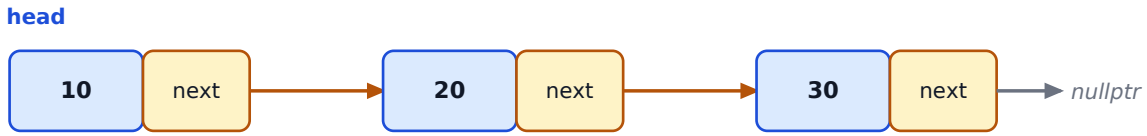


Figure 9.3 - A singly linked list: each node knows where the next one is

```
std::deque<int> dq;  dq.push_front(1); dq.push_back(2);
std::list<int> l{1, 2, 3};   l.push_front(0);   l.remove(2);   l.sort();
```

MY OWN NOTE

In practice `vector` beats `list` for almost everything, even insertions in the middle, because of CPU cache behaviour. Measure before choosing a linked list.

9.3 Associative containers: map and set

These keep elements sorted by key in a balanced tree. Lookup, insert and erase are O(log n).

```
#include <map>
#include <set>
std::map<std::string, int> age;
age["Asha"] = 21;                    // inserts or overwrites
age.insert({"Ben", 25});
age.emplace("Chi", 30);
age.at("Asha");                      // throws if missing
age["Zed"];                          // creates Zed with 0 !
if (auto it = age.find("Ben"); it != age.end())
    std::cout << it->first << " " << it->second;
age.contains("Chi");                 // C++20
age.erase("Ben");
for (const auto& [name, a] : age) std::cout << name << ":" << a << " ";   // sorted by name
```

```
std::set<int> s{5, 1, 3, 3, 1};      // unique + sorted: {1, 3, 5}
s.insert(4);  s.count(3);  s.lower_bound(2);
```

> **CAREFUL!**
>
> `map[key]` on a missing key **inserts** a default value. Use `find`, `contains` or `at` when you only want to look.

`multimap` and `multiset` allow duplicate keys.

9.4 Unordered containers (hash tables)

`unordered_map` and `unordered_set` use hashing. Average O(1) lookup, but no order. They are usually the fastest choice for "have I seen this?" or "count occurrences" problems.

```
#include <unordered_map>
std::unordered_map<std::string, int> freq;
for (const auto& w : words) ++freq[w];          // count words

std::unordered_set<int> seen;
for (int n : nums) if (!seen.insert(n).second)  // insert returns {iterator, wasInserted}
    std::cout << n << " is a duplicate\n";
```

	map / set	unordered_map / unordered_set
Structure	balanced tree (red-black)	hash table
Lookup	O(log n)	O(1) average, O(n) worst
Order	sorted by key	none
Key needs	`operator<`	hash function + `==`
Choose for	sorted output, range queries	fastest lookup

9.5 Container adaptors

```
#include <stack>
#include <queue>
std::stack<int> st;   st.push(1); st.top(); st.pop();            // LIFO
std::queue<int> q;    q.push(1); q.front(); q.pop();            // FIFO
std::priority_queue<int> maxHeap;   maxHeap.push(5); maxHeap.top();    // biggest first
std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap; // smallest first
```

9.6 pair, tuple, optional, variant

```
std::pair<std::string,int> p{"a", 1};   p.first;   p.second;
auto t = std::make_tuple(1, 2.5, "x");        std::get<1>(t);
auto [a, b, c] = t;                            // structured binding

std::optional<int> findIndex(...);             // value or nothing
if (auto r = findIndex(); r) std::cout << *r;
r.value_or(-1);

std::variant<int, std::string> v = "hi";      // exactly one of the types
std::get<std::string>(v);   std::holds_alternative<int>(v);
```

9.7 Choosing the right container

I need to...	Use
store a list and loop over it	`vector`
look up by key, order not needed	`unordered_map`
look up by key, keep sorted / find nearest	`map`
remove duplicates, check membership	`unordered_set` / `set`
add/remove at both ends	`deque`
always get the largest/smallest	`priority_queue`
fixed small size known at compile time	`array`
undo / DFS / matching brackets	`stack`
BFS / task queue	`queue`

Time complexity cheat sheet

	access	search	insert end	insert middle	erase
vector	O(1)	O(n)	O(1)*	O(n)	O(n)
deque	O(1)	O(n)	O(1)	O(n)	O(n)
list	O(n)	O(n)	O(1)	O(1)**	O(1)**
map/set	-	O(log n)	O(log n)	O(log n)	O(log n)
unordered_*	-	O(1) avg	O(1) avg	O(1) avg	O(1) avg

* amortised. ** once you already hold the iterator.

9.8 Chapter recap

- Start with `vector`; switch only when measurements or the access pattern demand it.
- `map` is sorted (tree), `unordered_map` is fast (hash).
- `map[key]` inserts; use `find` for lookup.
- Adaptors (`stack`, `queue`, `priority_queue`) restrict a container to one job.

CHAPTER 10

Iterators, Algorithms and Lambdas

Stop writing loops by hand

The STL separates what you store from what you do with it. Iterators are the glue, and algorithms are the toolbox.

10.1 Iterators

An iterator is an object that points at an element and can move to the next. It behaves like a pointer: `*it` gets the element, `++it` advances. Every container offers `begin()` (first element) and `end()` (one past the last).

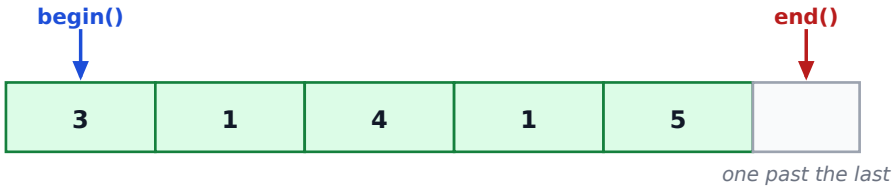


Figure 10.1 - A range is [begin, end): end points one past the last element

```
std::vector<int> v{3, 1, 4, 1, 5};
for (auto it = v.begin(); it != v.end(); ++it)
    std::cout << *it << ' ';

auto rit = v.rbegin();                // reverse iterator
auto cit = v.cbegin();                // const iterator: read-only
std::advance(it, 2);   std::distance(v.begin(), v.end());   // 5
auto n = std::next(v.begin(), 3);     // iterator 3 steps ahead
```

Category	Can do	Example
Input / Output	single pass read / write	`istream_iterator`
Forward	multi-pass, forward	`forward_list`
Bidirectional	also `--`	`list`, `map`, `set`
Random access	also `+n`, `-`, `<`	`vector`, `deque`, `array`
Contiguous (C++17)	random access + contiguous memory	`vector`, `array`, `string`

10.2 Lambdas

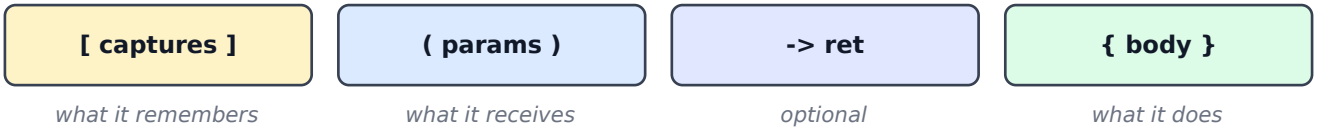


Figure 10.2 - The four parts of a lambda expression

```
auto add = [](int a, int b) { return a + b; };
add(2, 3);                                  // 5

int factor = 10;
auto scale = [factor](int x) { return x * factor; };      // capture by value
auto bump  = [&factor]()     { ++factor; };                // capture by reference
auto all   = [=]() { return factor; };                     // capture everything by value
auto allr  = [&]() { factor++; };                          // capture everything by reference

auto counter = [n = 0]() mutable { return ++n; };           // init-capture, mutable state
counter(); counter();                                       // 1, 2

auto generic = [](auto a, auto b) { return a + b; };        // C++14: works for any types
```

CAREFUL!

A lambda that captures by reference must not outlive the variables it refers to. Returning such a lambda from a function, or running it on another thread after the function ends, gives dangling references.

10.3 Algorithms

`<algorithm>` and `<numeric>` hold over 100 ready-made functions that take ranges of iterators. They are well tested, often faster than hand-written loops, and say *what* you mean.

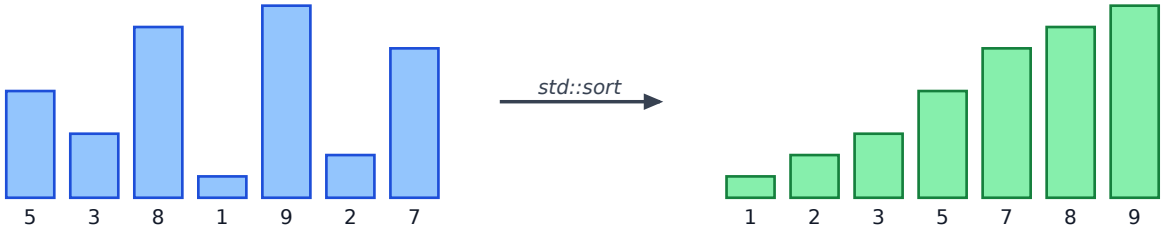


Figure 10.3 - std::sort rearranges a range into order

algorithms.cpp

```
#include <algorithm>
#include <numeric>
std::vector<int> v{5, 3, 8, 1, 9, 2, 7};

std::sort(v.begin(), v.end());                        // 1 2 3 5 7 8 9
std::sort(v.begin(), v.end(), std::greater<>());      // descending
std::sort(v.begin(), v.end(), [](int a, int b) { return abs(a-5) < abs(b-5); });

std::reverse(v.begin(), v.end());
auto it  = std::find(v.begin(), v.end(), 8);                  // iterator or end()
auto it2 = std::find_if(v.begin(), v.end(), [](int n){ return n > 6; });
std::count(v.begin(), v.end(), 1);
std::count_if(v.begin(), v.end(), [](int n){ return n % 2; });
std::binary_search(v.begin(), v.end(), 5);            // needs sorted data
auto lb = std::lower_bound(v.begin(), v.end(), 5);    // first >= 5
auto [mn, mx] = std::minmax_element(v.begin(), v.end());
std::accumulate(v.begin(), v.end(), 0);               // sum
std::all_of(v.begin(), v.end(), [](int n){ return n > 0; });
std::any_of(...);  std::none_of(...);
std::unique(v.begin(), v.end());
std::rotate(v.begin(), v.begin() + 2, v.end());
std::iota(v.begin(), v.end(), 1);                     // 1 2 3 4 ...
std::next_permutation(v.begin(), v.end());
```

transform, copy, for_each, partition

```
std::vector<int> in{1, 2, 3, 4}, out(in.size());
std::transform(in.begin(), in.end(), out.begin(), [](int n){ return n * n; });   // 1 4 9 16
std::vector<int> evens;
std::copy_if(in.begin(), in.end(), std::back_inserter(evens),
             [](int n){ return n % 2 == 0; });
std::for_each(in.begin(), in.end(), [](int n){ std::cout << n; });
std::partition(in.begin(), in.end(), [](int n){ return n < 3; });
```

10.4 Ranges (C++20)

Ranges let you pass a whole container instead of two iterators and chain operations lazily with `|`, much like a Unix pipeline.

```
#include <ranges>
namespace rv = std::views;

std::vector<int> nums{1, 2, 3, 4, 5, 6, 7, 8};
std::ranges::sort(nums);                       // no begin/end

auto result = nums | rv::filter([](int n){ return n % 2 == 0; })   // 2 4 6 8
                   | rv::transform([](int n){ return n * n; })    // 4 16 36 64
                   | rv::take(3);                                  // 4 16 36
for (int x : result) std::cout << x << ' ';

for (int i : rv::iota(1, 6)) std::cout << i;   // 12345
```

IN PLAIN WORDS

Views are lazy: nothing is computed until you iterate. They also do not own data, so the underlying container must stay alive.

10.5 Custom comparison and hashing

```
struct Person { std::string name; int age; };

// sort by age, then name
std::sort(people.begin(), people.end(), [](const Person& a, const Person& b) {
    return std::tie(a.age, a.name) < std::tie(b.age, b.name);
});

// a set ordered by a custom rule
auto byAge = [](const Person& a, const Person& b) { return a.age < b.age; };
std::set<Person, decltype(byAge)> s(byAge);

// custom hash for unordered_map
struct PHash {
    size_t operator()(const Person& p) const { return std::hash<std::string>{}(p.name); }
};
```

10.6 Chapter recap

- Iterators generalise pointers; a range is `[begin, end)`.
- Lambdas: `[capture](params){ body }`. Capture by value unless you need a reference and know the lifetime is safe.
- Prefer named algorithms (`sort`, `find_if`, `accumulate`, `transform`) to raw loops.
- C++20 ranges make pipelines readable: `v | filter(...) | transform(...)`.

CHAPTER 11

Memory Safety: RAII, Smart Pointers, Move Semantics

How modern C++ avoids leaks without a garbage collector

Modern C++ code rarely contains `new` or `delete`. Ownership is expressed in types, and cleanup is automatic.

11.1 RAII - Resource Acquisition Is Initialisation

The central idea of C++: tie the lifetime of a resource to the lifetime of an object. Acquire it in the constructor and release it in the destructor. Because destructors run on every exit path (normal return, early return, exception), cleanup can never be forgotten.

```
// BAD: manual
void f() {
    int* p = new int[1000];
    if (somethingWrong()) return;   // LEAK
    process(p);                     // may throw -> LEAK
    delete[] p;
}

// GOOD: RAII
void g() {
    std::vector<int> v(1000);       // freed automatically on every path
    if (somethingWrong()) return;
    process(v);
}

{ std::lock_guard<std::mutex> lock(m); /* critical section */ }   // unlocked at }
{ std::ifstream in("data.txt"); /* use */ }                       // file closed at }
```

REMEMBER THIS

RAII is how C++ manages memory, files, locks, sockets and database connections. If you remember one idea from these notes, make it this one.

11.2 unique_ptr - exclusive ownership

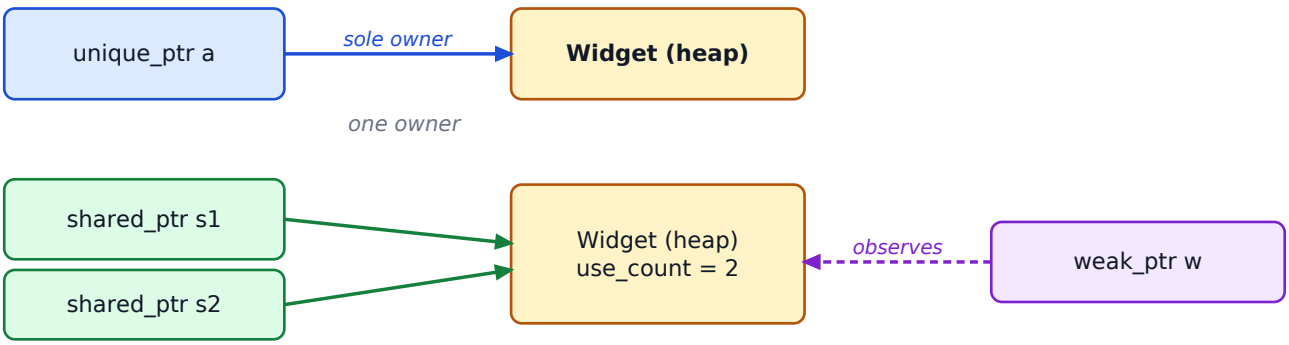


Figure 11.1 - unique_ptr has one owner; shared_ptr counts owners; weak_ptr only observes

```
#include <memory>
auto p = std::make_unique<Widget>(42);     // preferred over new
p->doThing();    (*p).doThing();    p.get();      // raw pointer, still owned by p
```

```
// auto q = p;                                  // ERROR: cannot copy
auto q = std::move(p);                          // ownership transferred; p is now null

std::unique_ptr<int[]> arr = std::make_unique<int[]>(100);   // array form

std::unique_ptr<Shape> make(bool c) {                        // factory returning ownership
    if (c) return std::make_unique<Circle>(1);
    return std::make_unique<Rect>(2, 3);
}
```

`unique_ptr` is the same size as a raw pointer and has no overhead. **It should be your default** whenever you need heap allocation or polymorphism.

11.3 shared_ptr and weak_ptr

```
auto a = std::make_shared<Widget>();     // use_count = 1
auto b = a;                                // use_count = 2: shared ownership
std::cout << a.use_count();                // 2
b.reset();                                 // count back to 1
// Widget is destroyed when the LAST shared_ptr goes away

std::weak_ptr<Widget> w = a;               // does not increase the count
if (auto locked = w.lock()) locked->doThing();   // null if already destroyed
```

CAREFUL!

Two objects holding `shared_ptr` to each other form a **cycle**: the counts never reach zero and both leak. Break cycles by making one direction a `weak_ptr` (for example, a child's pointer to its parent).

Smart pointer	Ownership	Cost	Use for
`unique_ptr`	exactly one owner	none	default choice, factories, polymorphic members
`shared_ptr`	many owners, counted	control block + atomic counts	truly shared lifetime (caches, graphs)
`weak_ptr`	none (observer)	small	breaking cycles, caches
raw `T*` / `T&`	none (non-owning)	none	borrowing for the duration of a call

TIP

Rule of thumb: owning raw pointers are bugs; non-owning raw pointers and references are fine. Pass `Widget&` or `Widget*` to functions that merely use the object, and `unique_ptr<Widget>` only to functions that take ownership.

11.4 Move semantics

Copying a large object duplicates all its data. Often the source is a temporary that is about to be thrown away anyway, so copying is wasteful. **Moving** transfers the internals (steals the pointer) and leaves the source empty but valid.

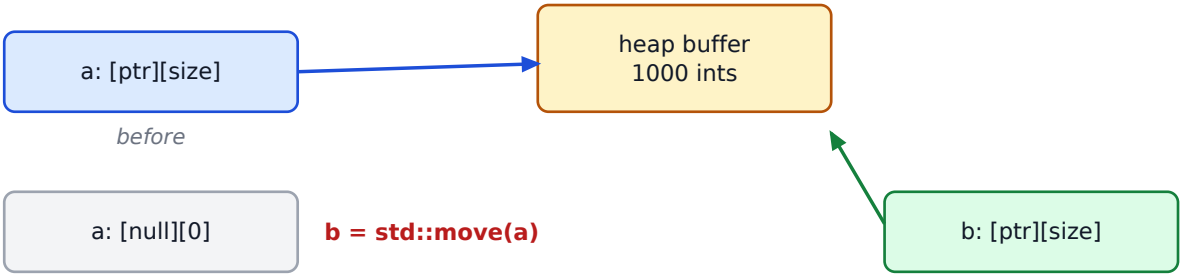


Figure 11.2 - A move steals the buffer pointer instead of copying 1000 ints

```
std::vector<int> a(1'000'000, 7);
std::vector<int> b = a;               // COPY: allocates and copies a million ints
std::vector<int> c = std::move(a);    // MOVE: just swaps a few pointers
// a is now valid but unspecified (in practice empty). Do not rely on its contents.

std::string s = "hello";
v.push_back(std::move(s));            // hand over s's buffer instead of copying

std::vector<std::string> make() {
    std::vector<std::string> r;
    r.push_back("x");
    return r;                         // moved (or elided) automatically; no std::move needed
}
```

lvalues, rvalues and forwarding

- An **lvalue** has a name and an address (`x`, `v[0]`). An **rvalue** is a temporary (`x + 1`, `f()`, `std::move(x)`).
- `T&&` in a concrete type binds to rvalues. In a deduced template parameter (`template<class T> void f(T&&)`) it is a *forwarding reference* that binds to anything.
- `std::move(x)` does not move anything. It only casts `x` to an rvalue so that a move constructor *can* be chosen.
- `std::forward<T>(x)` preserves whether the argument was an lvalue or rvalue when passing it on.

```
template <typename T, typename... Args>
std::unique_ptr<T> myMakeUnique(Args&&... args) {          // perfect forwarding
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}
```

11.5 Copy elision and the cost of returning

Since C++17, returning a prvalue (`return Widget{};`) is guaranteed not to copy or move at all. Returning a named local is usually optimised too (NRVO) or moved. The practical advice: return by value and do not fight the compiler.

11.6 Common memory bugs and how to find them

Bug	Symptom	Prevention
Memory leak	memory grows forever	smart pointers, containers
Dangling pointer/reference	random values, crashes	do not outlive the owner; no refs to locals
Buffer overflow	corruption, security hole	`vector::at`, `span`, `string`
Double free	crash in `free`	single owner (`unique_ptr`)
Uninitialised read	different results each run	initialise with `{}`

Tools

```
g++ -g -fsanitize=address,undefined -O1 main.cpp -o app   # catches most of these
valgrind --leak-check=full ./app                          # alternative on Linux
```

11.7 Chapter recap

- RAII: resources belong to objects; destructors clean up.
- `make_unique` by default, `make_shared` when sharing is real, `weak_ptr` to break cycles.
- Moving transfers ownership cheaply; `std::move` is just a cast.
- Run with AddressSanitizer while developing.

CHAPTER 12

Exceptions and File I/O

Handling things that go wrong, and reading and writing files

Real programs meet bad input, missing files and failed allocations. C++ gives you exceptions for the unexpected and streams for files.

12.1 Exceptions

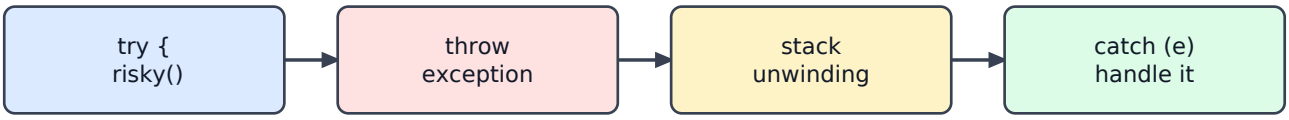


Figure 12.1 - A thrown exception unwinds the stack until a matching catch is found

exceptions.cpp

```
#include <stdexcept>
double divide(double a, double b) {
    if (b == 0) throw std::invalid_argument("division by zero");
    return a / b;
}

int main() {
    try {
        std::cout << divide(10, 0);
    } catch (const std::invalid_argument& e) {       // catch by const reference
        std::cerr << "bad argument: " << e.what() << "\n";
    } catch (const std::exception& e) {              // any standard exception
        std::cerr << "error: " << e.what() << "\n";
    } catch (...) {                                  // anything at all
        std::cerr << "unknown\n";
    }
}
```

OUTPUT

```
bad argument: division by zero
```

When `throw` runs, the current function stops, its local objects are destroyed (so RAII cleans up), and the search continues in the caller until a matching `catch` is found. If none is found, the program terminates.

The standard exception hierarchy

Exception	Typical cause
`std::exception`	base of all standard exceptions; has `what()`
`std::logic_error` / `invalid_argument` / `out_of_range`	bugs or bad arguments, e.g. `vector::at`
`std::runtime_error`	problems detected only at run time (file missing, parse failure)
`std::bad_alloc`	`new` could not get memory
`std::bad_cast`	failed `dynamic_cast` to a reference

Writing your own exception

```
class InsufficientFunds : public std::runtime_error {
    double shortBy_;
public:
    InsufficientFunds(double s) : std::runtime_error("insufficient funds"), shortBy_(s) {}
    double shortBy() const { return shortBy_; }
};

void withdraw(double amt) { if (amt > balance) throw InsufficientFunds(amt - balance); }
```

noexcept

`noexcept` promises a function will not throw. It lets the compiler optimise and is important for move operations: `std::vector` only uses your move constructor during growth if it is `noexcept`, otherwise it copies to stay safe.

```
void swapThem(int& a, int& b) noexcept;
MyType(MyType&&) noexcept;       // always mark move operations noexcept when true
```

Exception safety levels

Level	Guarantee
No-throw	never fails (destructors, swap, move)
Strong	if it fails, state is unchanged (commit or rollback)
Basic	no leaks, object still valid, but state may have changed

CAREFUL!

Never let an exception escape a destructor. Throwing while another exception is already unwinding calls `std::terminate`. Destructors are implicitly `noexcept`.

TIP

Use exceptions for exceptional, non-local failures. For expected outcomes ("not found") return `std::optional`, or in C++23 `std::expected<T, E>`.

12.2 File I/O

File streams work exactly like `cin`/`cout`: the same `<<` and `>>` operators. The streams close the file for you in their destructors, which is RAII again.

files.cpp

```
#include <fstream>

// writing
std::ofstream out("scores.txt");                // truncates; use std::ios::app to append
if (!out) { std::cerr << "cannot open\n"; return 1; }
out << "Asha " << 91 << "\n" << "Ben " << 78 << "\n";

// reading word by word
std::ifstream in("scores.txt");
std::string name; int score;
while (in >> name >> score)
    std::cout << name << " scored " << score << "\n";

// reading line by line
std::ifstream f("notes.txt");
std::string line;
while (std::getline(f, line))
    std::cout << line << "\n";
```

Mode flag	Meaning
`std::ios::in` / `out`	read / write
`std::ios::app`	append to the end
`std::ios::trunc`	empty the file on open (default for `ofstream`)
`std::ios::binary`	no newline translation; for raw bytes

Binary files

```
struct Record { int id; double value; };          // plain data only
Record r{7, 3.5};
std::ofstream bo("data.bin", std::ios::binary);
bo.write(reinterpret_cast<const char*>(&r), sizeof r);

std::ifstream bi("data.bin", std::ios::binary);
Record r2;
bi.read(reinterpret_cast<char*>(&r2), sizeof r2);
```

Checking stream state

```
if (in.fail())  { /* last operation failed */ }
if (in.eof())   { /* reached end of file */ }
in.clear();                             // reset error flags
in.seekg(0);                            // go back to the start
```

std::filesystem (C++17)

```
#include <filesystem>
namespace fs = std::filesystem;
fs::exists("a.txt");
fs::create_directories("out/logs");
fs::file_size("a.txt");
for (const auto& e : fs::directory_iterator("."))
    std::cout << e.path().filename() << "\n";
fs::remove("old.tmp");
```

12.3 String streams and formatting

```
#include <sstream>
std::ostringstream os;
os << "x=" << 5 << ", y=" << 2.5;
std::string result = os.str();                  // "x=5, y=2.5"

// C++20 std::format and C++23 std::print
#include <format>
std::string s = std::format("{} scored {:>5.1f}%", "Asha", 91.25);   // "Asha scored  91.2%"
```

12.4 Worked example: a word-frequency counter

wordfreq.cpp

```
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include <vector>
#include <algorithm>
#include <cctype>

int main(int argc, char** argv) {
    if (argc < 2) { std::cerr << "usage: wc file\n"; return 1; }
    std::ifstream in(argv[1]);
    if (!in) { std::cerr << "cannot open " << argv[1] << "\n"; return 1; }

    std::map<std::string, int> freq;
    std::string w;
    while (in >> w) {
        w.erase(std::remove_if(w.begin(), w.end(),
                [](unsigned char c){ return !std::isalnum(c); }), w.end());
        std::transform(w.begin(), w.end(), w.begin(),
                [](unsigned char c){ return std::tolower(c); });
        if (!w.empty()) ++freq[w];
    }

    std::vector<std::pair<std::string,int>> v(freq.begin(), freq.end());
    std::sort(v.begin(), v.end(), [](auto& a, auto& b){ return a.second > b.second; });
    for (size_t i = 0; i < std::min<size_t>(5, v.size()); ++i)
        std::cout << v[i].first << ": " << v[i].second << "\n";
}
```

This one short program uses streams, a map, a vector of pairs, algorithms and lambdas together, which is typical of idiomatic C++.

12.5 Chapter recap

- `throw` by value, `catch` by `const&`; the standard hierarchy gives you `what()`.
- RAII makes exceptions safe: destructors run during unwinding.
- Mark move operations and swaps `noexcept`.
- `ifstream` / `ofstream` use the same operators as `cin` / `cout` and close themselves.
- Always check that a file opened.

CHAPTER 13

Modern C++ (11 to 23) at a Glance

The features that changed how C++ is written

If your notes stop at C++98 they miss most of what makes the language pleasant today. Here is a tour, each feature with a tiny example.

13.1 C++11: the big reset

```
auto x = 5;                              // type deduction
for (auto& e : v) { }                    // range-based for
int* p = nullptr;                        // not NULL or 0
std::vector<int> v{1, 2, 3};             // uniform initialisation
auto f = [](int a) { return a * 2; };    // lambda
std::unique_ptr<T>, std::shared_ptr<T>   // smart pointers
T(T&&) noexcept;                         // move semantics
static_assert(sizeof(int) == 4, "int must be 32-bit");
using Vec = std::vector<int>;            // alias (better than typedef)
enum class Color { R, G, B };
constexpr int sq(int x) { return x * x; }
std::thread t(work);                     // threads (Chapter 14)
class A { A() = default; A(const A&) = delete; };   // = default and = delete
void g() override;  class B final {};
std::array, std::unordered_map, std::tuple, std::function, std::chrono
```

13.2 C++14

```
auto add = [](auto a, auto b) { return a + b; };    // generic lambda
auto make() { return std::make_unique<int>(1); }       // return type deduction
std::make_unique<T>(...);                              // finally!
int big = 1'000'000;   int bits = 0b1010'1010;        // digit separators, binary literals
using namespace std::literals;   auto s = "hi"s;   auto d = 500ms;
```

13.3 C++17

```
auto [a, b] = std::pair{1, 2};                    // structured bindings
for (const auto& [k, v] : myMap) { }

if (auto it = m.find(k); it != m.end()) { }           // if with initialiser
if constexpr (std::is_integral_v<T>) { }              // compile-time branch

std::optional<int> o;   std::variant<int,string> v;   std::any a;
std::string_view sv = "read only view";
std::filesystem::path p = "/tmp";
inline constexpr double kPi = 3.14159;                // inline variables
template <typename... A> auto sum(A... a) { return (a + ...); }   // fold expressions
std::vector v2{1, 2, 3};                              // CTAD
[[nodiscard]] int compute();   [[maybe_unused]] int x;   [[fallthrough]];
std::for_each(std::execution::par, v.begin(), v.end(), f);        // parallel algorithms
```

13.4 C++20: the second big reset

Feature	What it gives you
Concepts	named, checked requirements on templates
Ranges and views	composable `v | filter | transform` pipelines
Coroutines	`co_await`, `co_yield`, `co_return` for generators and async code
Modules	`import std;` replaces header inclusion; faster builds, no macros leaking
Three-way comparison	`<=>` and `= default` generate all comparison operators
std::format	type-safe, Python-style formatting
`std::span`	non-owning view over contiguous memory
designated initialisers	`Point p{.x = 1, .y = 2};`
`consteval` / `constinit`	must-be-compile-time functions and variables
`std::jthread`	thread that joins automatically and supports cancellation

```
// span: safe, cheap view that works with arrays, vectors, std::array
void print(std::span<const int> s) { for (int x : s) std::cout << x << ' '; }
int raw[] = {1, 2, 3};   std::vector<int> v{4, 5};
print(raw);  print(v);

// three-way comparison
struct Version {
    int major, minor;
    auto operator<=>(const Version&) const = default;   // gives <, >, <=, >=, ==, !=
};
Version{1, 2} < Version{1, 10};       // true

// coroutine generator (needs a library type such as std::generator in C++23)
// std::generator<int> range(int n) { for (int i = 0; i < n; ++i) co_yield i; }
```

13.5 C++23

```
std::print("Hello, {}!\n", name);                  // finally a sane print
std::expected<int, std::string> parse(std::string_view s);   // value or error, no exceptions
auto r = parse("42");
if (r) use(*r); else log(r.error());

std::flat_map<K,V>;   std::mdspan;   std::generator<T>;
if consteval { }        // different code at compile time vs run time
std::unreachable();     // optimiser hint
auto&& x = std::views::zip(a, b);   // iterate two ranges together
```

13.6 Old style vs modern style

Old	Modern
`int* p = new int[n];` … `delete[] p;`	`std::vector<int> v(n);`
`NULL`	`nullptr`
`typedef std::map<int,int> M;`	`using M = std::map<int,int>;`
`for (it = v.begin(); it != v.end(); ++it)`	`for (const auto& x : v)`
`std::auto_ptr`	`std::unique_ptr`
functor class for sort	a lambda
`#define N 100`	`constexpr int N = 100;`
plain `enum`	`enum class`
`char*` and `strcpy`	`std::string` / `std::string_view`
`bind1st` etc.	lambda
raw `new` in `main`	`make_unique`
`const char*` out params	return `std::string` / `optional`

MY OWN NOTE

You do not need to learn every feature. A good starting set is: `auto`, range-for, `unique_ptr`, lambdas, `vector`/`map`, `optional`, structured bindings, `string_view`, and (if your compiler supports it) concepts and ranges.

13.7 Chapter recap

- C++11 changed the style of the language; C++20 changed it again (concepts, ranges, modules, coroutines).
- Use the newest standard your toolchain supports (`-std=c++20` or `-std=c++23`).
- Modern C++ means fewer raw pointers, fewer manual loops, more types that say what they mean.

CHAPTER 14

Concurrency and Performance

Doing several things at once, and doing each thing fast

Since C++11 threads, locks and atomics are part of the language. Concurrency is powerful and easy to get wrong, so the notes here focus on the safe patterns.

14.1 Threads

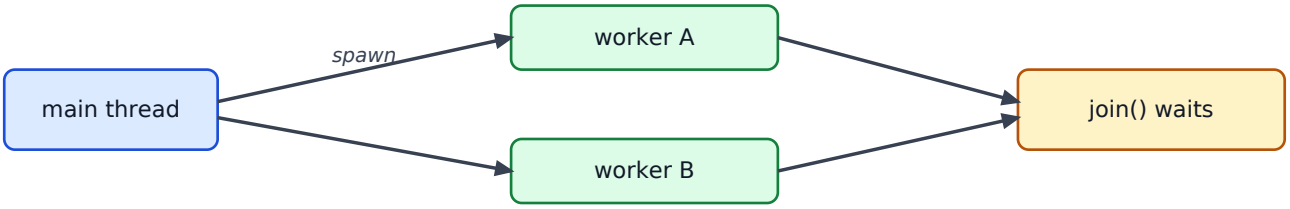


Figure 14.1 - Spawn threads, then join them to wait for completion

threads.cpp

```
#include <thread>
#include <iostream>

void work(int id) { std::cout << "worker " << id << "\n"; }

int main() {
    std::thread a(work, 1);                       // starts running immediately
    std::thread b([]{ std::cout << "lambda\n"; });
    a.join();                                     // wait for a
    b.join();
}                                                 // compile with -pthread
```

CAREFUL!

A `std::thread` that is destroyed while still joinable calls `std::terminate`. Always `join()` or `detach()` it, or use `std::jthread` (C++20), which joins automatically.

14.2 Data races and mutexes

If two threads touch the same variable and at least one writes, without synchronisation, you have a **data race**, which is undefined behaviour. Protect shared data with a mutex.

```
#include <mutex>
int counter = 0;
std::mutex m;

void add() {
    for (int i = 0; i < 100000; ++i) {
        std::lock_guard<std::mutex> lock(m);     // RAII: unlocks at end of scope
        ++counter;
    }
}
// two threads running add(): counter is exactly 200000 with the mutex,
// some unpredictable smaller number without it.
```

```
std::scoped_lock both(m1, m2);                  // lock several mutexes without deadlock
```

Atomics

```
#include <atomic>
std::atomic<int> hits{0};
void work() { for (int i = 0; i < 100000; ++i) ++hits; }   // lock-free, thread-safe
hits.load();   hits.store(5);   hits.fetch_add(2);
hits.compare_exchange_strong(expected, desired);
```

14.3 Condition variables

```
std::mutex m;  std::condition_variable cv;  std::queue<int> q;  bool done = false;

void producer() {
    for (int i = 0; i < 5; ++i) { { std::lock_guard l(m); q.push(i); } cv.notify_one(); }
    { std::lock_guard l(m); done = true; }  cv.notify_all();
}
void consumer() {
    std::unique_lock lk(m);
    while (true) {
        cv.wait(lk, [] { return !q.empty() || done; });
        while (!q.empty()) { std::cout << q.front(); q.pop(); }
        if (done) break;
    }
}
```

14.4 async, future, promise

```
#include <future>
long long sumRange(const std::vector<int>& v, size_t lo, size_t hi) {
    return std::accumulate(v.begin() + lo, v.begin() + hi, 0LL);
}

auto f1 = std::async(std::launch::async, sumRange, std::cref(v), 0, v.size() / 2);
auto f2 = std::async(std::launch::async, sumRange, std::cref(v), v.size() / 2, v.size());
long long total = f1.get() + f2.get();
```

`std::async` is the easiest way to run a function in the background and collect its result. For a manual hand-off between threads use `std::promise`/`std::future`.

Tool	Use it for
`thread` / `jthread`	run a function concurrently
`mutex` + `lock_guard`/`scoped_lock`	protect shared data
`atomic<T>`	single shared counters/flags
`condition_variable`	wait until something happens
`async` + `future`	run a task, get a result
`std::execution::par`	parallel versions of algorithms (C++17)

REMEMBER THIS

Rules for safe concurrency: share as little as possible, prefer immutable data, always lock with RAII wrappers, always lock mutexes in the same order, and never call unknown code while holding a lock.

14.5 Writing fast C++

- **Measure first.** Use a profiler (`perf`, VTune, Instruments) and a benchmark library; guesses are usually wrong.
- **Choose the right algorithm and container** before micro-optimising: O(n log n) beats O(n^2) every time.
- **Compile with optimisation:** `-O2` or `-O3`. A debug build can be 10x slower.
- **Avoid needless copies:** pass `const&`, use `move`, `reserve` vectors, use `emplace_back`.
- **Think about cache:** contiguous data (`vector`) is much faster than pointer-chasing (`list`, `map`) because the CPU loads neighbouring bytes together.
- **Avoid allocation in hot loops;** reuse buffers.
- **Prefer `'\n'` to `std::endl`** and turn off stdio sync for heavy I/O.

Timing code

```
#include <chrono>
auto t0 = std::chrono::steady_clock::now();
run();
auto t1 = std::chrono::steady_clock::now();
auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(t1 - t0).count();
std::cout << ms << " ms\n";
```

14.6 Chapter recap

- Join your threads (or use `jthread`). Protect shared writes with a mutex held by a RAII lock.
- `atomic` for simple counters; `async`/`future` for tasks with results.
- Measure, then optimise; algorithms and data layout matter most.

CHAPTER 15

Best Practices, Mistakes and Interview Questions

Habits that keep C++ code correct, plus a revision section

Here the rest of the book comes together as a checklist, a list of traps, and the questions interviewers ask most often.

15.1 The habits of good C++ programmers

1. **Use RAII for everything** that must be released. No naked `new`/`delete`.
2. **Default to `std::vector`, `std::string`, `std::unique_ptr`.**
3. **Make things `const` by default** (variables, parameters, methods). Remove `const` only when needed.
4. **Initialise every variable** at the point of declaration, preferably with `{}`.
5. **Prefer algorithms and range-for** to index loops.
6. **Use references or `string_view` for read-only parameters.**
7. **Use `override`** on every overriding function, and a `virtual` destructor on every polymorphic base.
8. **Prefer `enum class`, `constexpr`, `using`** over `enum`, `#define`, `typedef`.
9. **Enable warnings** (`-Wall -Wextra -Wpedantic`) and treat them as errors in CI.
10. **Run sanitizers** (`-fsanitize=address,undefined`) and static analysis (`clang-tidy`) regularly.
11. **Follow the Rule of Zero.**
12. **Keep headers light:** include what you use, forward-declare when possible, never `using namespace` in headers.

15.2 Header files, include guards, and organising a project

Three files

```
// shape.h
#pragma once                        // or #ifndef SHAPE_H / #define SHAPE_H / #endif
#include <string>

class Shape {
public:
    explicit Shape(std::string name);
    double area() const;
private:
    std::string name_;
};

// shape.cpp
#include "shape.h"
Shape::Shape(std::string name) : name_(std::move(name)) {}
double Shape::area() const { return 0; }

// main.cpp
#include "shape.h"
int main() { Shape s("blob"); }
```

Building

```
g++ -std=c++20 -Wall -Wextra -c shape.cpp   # shape.o
g++ -std=c++20 -Wall -Wextra -c main.cpp    # main.o
g++ shape.o main.o -o app                   # link

# or with CMake (CMakeLists.txt)
cmake_minimum_required(VERSION 3.20)
project(demo CXX)
set(CMAKE_CXX_STANDARD 20)
add_executable(app main.cpp shape.cpp)
```

IN PLAIN WORDS

The one-definition rule: a function or class may be *declared* many times (headers) but *defined* exactly once across the whole program. Defining a non-inline function in a header and including it from two `.cpp` files gives a "multiple definition" linker error.

15.3 Common mistakes, quick fixes

Mistake	Why it bites	Fix
Reading an uninitialised variable	garbage, varies per run	`int x{};`
`=` instead of `==` in `if`	assigns, always true	`-Wall`, careful reading
Off-by-one in loops (`<=` vs `<`)	out-of-bounds access	range-for, `at()`
Integer division	`1/2 == 0`	use a `double` operand
Comparing signed and unsigned	negative becomes huge	`static_cast`, same types
Returning reference to a local	dangling	return by value
Forgetting `virtual` destructor	leaks, UB on delete	`virtual ~Base() = default;`
Slicing	derived part lost	use pointers/references
Iterator invalidation	crash after `push_back`/`erase`	re-fetch iterators, use indices
`map[key]` just to read	inserts default	`find` / `contains`
Copying big objects	slow	`const&`, `move`
Mixing `new[]` with `delete`	undefined behaviour	use `vector`
Not `join`ing a thread	terminate	`jthread`
Using `std::endl` in loops	flushes every line	`'\n'`
Macros with side effects	`MAX(a++, b)` evaluates twice	templates / `constexpr`

15.4 Interview questions with short answers

Language basics

- **Difference between C and C++?** C++ adds classes, templates, references, exceptions, namespaces, the STL, RAII and stronger typing on top of (mostly) C.
- **Pointer vs reference?** A pointer can be null and re-seated; a reference is an alias, always valid, bound once.
- `const int*` **vs** `int* const`**?** First: data is const. Second: the pointer is const.
- **What is** `nullptr`**?** A typed null pointer constant that replaces `NULL`/`0` and avoids overload ambiguity.
- `struct` **vs** `class`**?** Only default access (public vs private).

- `new` **vs** `malloc`**?** `new` calls constructors and returns the right type; `malloc` returns raw bytes. Match `new`/`delete`, `malloc`/`free`.
- **What is undefined behaviour?** Code the standard does not define; the compiler may do anything (crash, wrong output, seemingly work).
- **Stack vs heap?** Automatic, fast, small vs manual, slower, large.

OOP

- **What is polymorphism?** Same interface, different behaviour. Run-time via `virtual`; compile-time via overloading and templates.
- **Why a virtual destructor?** So deleting through a base pointer runs the derived destructor.
- **What is a pure virtual function?** `= 0`; makes the class abstract.
- **Overloading vs overriding?** Overloading: same name, different parameters, chosen at compile time. Overriding: derived class replaces a virtual function, chosen at run time.
- **What is the vtable?** Per-class table of virtual function addresses; each object has a vptr to it.
- **Shallow vs deep copy?** Shallow copies the pointer; deep copies the pointee. Classes owning resources need deep copy (or move-only).
- **Rule of 3/5/0?** See 6.4.
- **What does** `explicit` **do?** Stops implicit conversions through a single-argument constructor.

Modern C++ and STL

- `unique_ptr` **vs** `shared_ptr`**?** Sole vs shared ownership; `unique_ptr` is free, `shared_ptr` costs a control block and atomic counts.
- **What does** `std::move` **do?** Casts to an rvalue reference; the move constructor does the actual transfer.
- `vector` **vs** `list`**?** `vector` is contiguous and cache friendly; `list` has O(1) splice but slow traversal.
- `map` **vs** `unordered_map`**?** Sorted tree O(log n) vs hash O(1) average.
- **What is RAII?** Resource lifetime bound to object lifetime.
- **What is a lambda capture?** The list in `[]` of outside variables the lambda can use, by value or by reference.
- **What is** `auto`**?** Compiler-deduced type, still static.
- `constexpr` **vs** `const`**?** `constexpr` must be usable at compile time.
- **What is iterator invalidation?** When a container change makes existing iterators/references unusable.

15.5 Practice problems (with solution sketches)

#	Problem	Hint
1	Reverse a string without `std::reverse`	two indexes moving inward, `swap`
2	Check if a string is a palindrome	compare `s[i]` with `s[n-1-i]`
3	Find the two numbers in an array that sum to a target	`unordered_map<value,index>`, one pass
4	Count the frequency of each word in a file	see section 12.4
5	Remove duplicates from a vector, keep order	`unordered_set` + `erase`/`remove_if`
6	Implement a `Stack<T>` class template	section 8.2 with `vector`
7	Write a linked list with `unique_ptr` nodes	`struct Node { int v; unique_ptr<Node> next; };`
8	Matching brackets in an expression	`stack<char>`
9	Shape hierarchy with total area	`vector<unique_ptr<Shape>>` and a virtual `area()`
10	Producer/consumer queue	mutex + condition_variable

solutions.cpp

```
// Solution sketch for #3: two-sum in O(n)
std::vector<int> twoSum(const std::vector<int>& a, int target) {
    std::unordered_map<int,int> seen;                 // value -> index
    for (int i = 0; i < (int)a.size(); ++i) {
        if (auto it = seen.find(target - a[i]); it != seen.end())
            return {it->second, i};
        seen[a[i]] = i;
    }
    return {};
}

// Solution sketch for #8: matching brackets
bool balanced(const std::string& s) {
    std::stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else if (c == ')' || c == ']' || c == '}') {
            if (st.empty()) return false;
            char o = st.top(); st.pop();
            if ((c == ')' && o != '(') || (c == ']' && o != '[') ||
                (c == '}' && o != '{')) return false;
        }
    }
    return st.empty();
}
```

15.6 One-page cheat sheet

Task	Code
Print	`std::cout << x << '\n';`
Read a line	`std::getline(std::cin, s);`
Vector of n zeros	`std::vector<int> v(n);`
Sort	`std::sort(v.begin(), v.end());`
Sort descending	`std::sort(v.rbegin(), v.rend());`
Sum	`std::accumulate(v.begin(), v.end(), 0);`
Max element	`*std::max_element(v.begin(), v.end());`
Find	`std::find(v.begin(), v.end(), x) != v.end()`
Count words	`++freq[word];` with `unordered_map<string,int>`
Convert	`std::stoi(s)`, `std::to_string(n)`
Swap	`std::swap(a, b);`
Min / max	`std::min(a, b)`, `std::max(a, b)`
New object	`auto p = std::make_unique<T>(args);`
Lambda	`[&](int x){ return x * k; }`
Open file	`std::ifstream in(path); if (!in) ...`
Random number	`std::mt19937 g{std::random_device{}()};` `std::uniform_int_distribution<int> d(1, 6); d(g);`
Time it	`std::chrono::steady_clock::now()`
Compile	`g++ -std=c++20 -Wall -Wextra -O2 -fsanitize=address,undefined x.cpp`

15.7 Where to go next

- Practise: solve problems on LeetCode, Codeforces, AtCoder or Advent of Code in C++.
- Read: *A Tour of C++* (Stroustrup), *Effective Modern C++* (Meyers), *C++ Primer* (Lippman), the C++ Core Guidelines.
- Reference: cppreference.com is the best online reference; Compiler Explorer (godbolt.org) shows the assembly your code becomes.
- Build something: a text-based game, a tiny shell, a chat client/server, a JSON parser. Real projects teach what notes cannot.

MY OWN NOTE

Do not try to memorise everything above. Write code, make the mistakes from section 15.3 yourself once, and the rules will stick. Then come back to these notes as a reference.